



BACHELOR'S THESIS

Higher-Order Message Passing in Neural Networks

A Survey and an Experimental Study

Adrián Ruiz Doblas

BSc in Mathematics

Work carried out at: University of California, Los Angeles

Supervised by: Prof. Guido Montúfar

Academic Year 2025–2026

“There does however seem to be some undefinable sense that a certain piece of mathematics is ‘on to something’, that it is a piece of a larger puzzle waiting to be explored further.”

Terence Tao, What is good mathematics? (2007)

Acknowledgments

I would like to express my most sincere gratitude to Prof. Guido Montúfar, who proposed the topic of this thesis and trusted me to develop a project that required a considerable degree of independence and flexibility. His support made it possible for me to pursue this work under circumstances that involved important academic and personal decisions, and I am grateful for that trust. I am also thankful for the opportunity to take part in the Mathematical Machine Learning group at UCLA, to present parts of this thesis there, and to receive critical and constructive feedback. Those discussions have been especially valuable in helping me mature during these first steps into mathematical research.

I would also like to thank my friends and classmates who have accompanied me throughout these five years. Many of the memories I will keep from this period come not only from the university itself, but from the trips, everyday moments, long conversations, and shared routines that gradually became an essential part of these years. I am especially grateful to those who became true friends along the way, making even the most demanding stages lighter and more meaningful.

Finally, I am deeply grateful to my family for their unconditional support throughout these years. I am especially thankful for the trust they have placed in me at every step of this degree, and for supporting the decisions that have gradually shaped my academic path. This was especially important when I decided to spend part of my studies in the United States, and later when this project emerged there and I had the opportunity to stay longer than originally planned in order to carry it out. I am also grateful for their patience, encouragement, and willingness to listen to me speak about mathematics, research, and this work, even when the details were far from easy to follow. Their confidence has given me the stability and freedom needed to reach this point.

Abstract

This thesis develops a self-contained introduction to higher-order message passing, progressing from graphs and ordinary message passing neural networks through the higher-order structures used in graph learning (hypergraphs, simplicial, cellular, and combinatorial complexes) and random-walk propagation on graphs and hypergraphs, to the formulation and experimental validation of a new hypergraph-derived message passing layer. The proposed EO-Pattern layer keeps a pairwise, vertex-centered aggregation support but conditions each message on two quantities computed from the hypergraph incidence structure: the equal-edges propagation kernel P^{EE} , derived from a random walk on hyperedge connectivity, and a cardinality-pattern descriptor χ_{vu} summarizing the sizes of hyperedges shared by adjacent vertices. We prove that every non-isolated row of the propagation kernel is stochastic and that the layer is permutation-equivariant, and, through two structural propositions, characterize regimes in which P^{EE} collapses to uniform neighborhood averaging or χ_{vu} loses pair-dependent variation. Three experiments test the construction: a synthetic twin task shows that χ_{vu} recovers cardinality information that ordinary clique expansion provably destroys; a real-world co-citation benchmark (Cora, Citeseer) yields no observable benefit from χ_{vu} and at most a small gain from P^{EE} over uniform neighborhood aggregation; and a constructive gadget exhibits a regime in which the two quantities are complementary, with their joint signature uniquely identifying an informative neighbor that neither normalized quantity identifies alone. Together, these results delineate some regimes in which hypergraph-derived structural information helps or becomes inert, and motivate falsifiable mutual-information-based diagnostics whose ability to predict usefulness on new data remains to be tested.

Keywords: higher-order message passing, hypergraphs, random walks, graph neural networks.

Contents

1	Introduction	1
1.1	From Graphs to Higher-Order Relational Structure	1
1.2	From Message Passing to Higher-Order Propagation	1
1.3	Problem Statement and Objectives	2
2	Graphs and Message Passing	4
2.1	Graphs, Features, and Permutation Symmetry	4
2.2	Message Passing Neural Networks	6
2.3	Canonical Examples: GCN and GIN	7
3	Higher-Order Structures and Propagation Operators	9
3.1	Why Higher-Order Structures?	9
3.2	Hypergraphs	9
3.3	Beyond Hypergraphs: Simplicial, Cellular, and Combinatorial Complexes	10
3.4	Message Passing as an Operator Choice	11
3.5	Random walk with restart and PageRank	12
3.6	Random Walks and Diffusion on Hypergraphs	13
4	The EO-Pattern MPNN	16
4.1	Motivation: What Clique Expansions Forget	16
4.2	EO-Inspired Cardinality-Pattern Message Passing	16
5	Experiments and Validation	25
5.1	Synthetic Hypergraph Classification Task	25
5.2	Real-World Benchmark: Co-citation Node Classification	27
5.3	Joint Usefulness of P^{EE} and χ : A Constructive Gadget	28
5.4	When Do the Hypergraph-Derived Quantities Carry Signal?	30
6	Discussion, Limitations, and Future Work	33
7	Conclusions	35
	References	36
	Appendices	38
A	Experimental Protocol and Reproducibility Details	38
A.1	Datasets and experimental settings	38
A.2	Controlled data-generation details	39
A.3	Randomness and ablation protocol	39
A.4	Accuracy and macro-F1	40

1. Introduction

1.1. From Graphs to Higher-Order Relational Structure

Graphs are among the simplest mathematical objects for representing relational structure. A set of vertices describes the entities of a system, while edges encode which pairs of entities are related. This minimal language has proved remarkably flexible: social interactions, molecules, citation networks, communication systems, and many other relational datasets can all be represented, at least at a first level, as graphs. In machine learning, this observation has led to the development of Graph Neural Networks (GNNs), where the graph is not merely auxiliary information, but the domain on which the model operates (Kipf and Welling, 2017; Wu et al., 2021).

The central mechanism behind many GNNs is message passing: each vertex carries a representation, receives information from its neighbors, aggregates the received messages, and updates its own state. This idea was formalized by Gilmer et al. (2017) through the framework of Message Passing Neural Networks (MPNNs), which provides a common language for a large family of graph-based architectures. In this view, a neural network on a graph is built from three ingredients: a set of carrier objects, usually the vertices; a notion of neighborhood, usually adjacency; and a rule for propagating and aggregating information. Different architectures can then be understood as different choices of propagation, aggregation, and update operators (Xu et al., 2019).

This graph-based viewpoint is the starting point of the present work. However, it also reveals a natural limitation. Graphs encode relations between pairs of vertices, but many systems contain interactions that are not intrinsically pairwise: a paper written by three authors is not the same object as three independent pairwise collaborations; a chemical, biological, social, or computational interaction may involve a whole group of entities at once. When such a relation is projected onto an ordinary graph, part of the original structure may be lost. Distinct higher-order configurations can induce the same pairwise shadow.

This motivates the passage from graphs to higher-order structures: hypergraphs, simplicial complexes, cellular complexes, and combinatorial complexes extend the graph formalism by allowing relations involving more than two elements, or relations living across several ranks or dimensions. These structures have become increasingly relevant in graph learning and topological deep learning because they provide richer domains on which relational data can be represented and processed (Hajij et al., 2023; Besta et al., 2026; Papillon et al., 2025). From this perspective, the central question is no longer only how to perform message passing on a graph, but what message passing should mean when the underlying relational object is not purely pairwise.

1.2. From Message Passing to Higher-Order Propagation

Moving from graphs to higher-order structures changes the nature of the modelling problem. In an ordinary graph, the basic carrier objects and neighborhoods are usually clear: vertices exchange messages along edges. In a higher-order domain, several choices become possible. Representations may be attached to vertices, hyperedges, simplices, cells, or objects of several ranks. Messages may travel through incidence relations, boundary and coboundary relations, adjacency relations, or diffusion operators. Thus, higher-order message passing is not just graph message passing applied to a larger object, but it also requires deciding what the relevant objects are and which geometry of propagation the model should follow.

A useful way to organize this landscape is the operator viewpoint. In many common graph neural network architectures, a message passing layer can be interpreted as a propagation operator followed by a learnable transformation and a nonlinearity (Kipf and Welling, 2017; Wu et al., 2021). In the graph case, such operators are often related to adjacency matrices, Laplacians, or random-walk kernels. In higher-order domains, the same principle remains useful, but the possible operators become richer. Random walks on hypergraphs, for example, do not have a unique canonical form: one must specify how a walker selects an incident hyperedge, how it moves within it, and how hyperedge cardinalities influence the transition probabilities (Carletti et al., 2020). These choices determine what structural information is made available to the neural network.

This thesis follows precisely this conceptual route. It starts from the classical graph setting, introduces message passing as the fundamental mechanism behind GNNs, studies the higher-order domains used in graph learning, and then examines random walks and diffusion operators as possible propagation mechanisms beyond ordinary graphs. The aim is both expository and constructive: to enter the field of higher-order message passing in a self-contained way, and then to use that understanding to formulate a concrete and original hypergraph-derived message passing layer.

The proposal developed in the thesis is motivated by a specific tension in the literature. On the one hand, relational and auxiliary-graph constructions provide powerful general frameworks for describing message passing on topological or higher-order domains (Taha et al., 2025). On the other hand, if a hypergraph is immediately replaced by an auxiliary graph, one must ask which parts of the original hypergraph structure remain visible to the model. In particular, hyperedge incidence and hyperedge cardinality may carry information that is not present in an unweighted clique expansion. The guiding idea of the proposed construction is therefore to keep a vertex-centered MPNN structure, while conditioning pairwise messages on quantities computed directly from the hypergraph incidence data.

1.3. Problem Statement and Objectives

The central problem addressed in this work is the following: can one construct a vertex-centered message passing paradigm for hypergraphs in which the messages are still exchanged between vertices, but are conditioned by genuinely hypergraph-derived information, so that the model is sensitive to incidence and cardinality patterns that would be invisible in a purely pairwise graph projection?

More concretely, this thesis studies whether a pairwise MPNN can be enriched with hypergraph-derived structural quantities, such as an equal-edges propagation kernel and a cardinality-pattern descriptor, in a way that distinguishes higher-order configurations that ordinary clique-based graph baselines cannot distinguish. The goal is not to claim a universal replacement for existing hypergraph neural networks, but to isolate and test a minimal construction showing how higher-order information can enter a vertex-centered message passing layer.

Hence, the objectives of the project are the following:

1. To build a gradual and self-contained introduction to graph neural networks, starting from graphs and standard message passing.
2. To survey the main higher-order structures used in graph learning, including hypergraphs,

simplicial complexes, cellular complexes, and combinatorial complexes.

3. To understand higher-order message passing through an operator-centered viewpoint, with special attention to random walks and diffusion-based propagation mechanisms.
4. To formulate a hypergraph-derived message passing layer inspired by incidence and cardinality-sensitive structural information, situated between ordinary clique-expansion baselines and more general relational message passing frameworks.
5. To validate the proposal experimentally through three complementary settings: a synthetic twin task isolating the information lost through clique expansion, real-world co-citation benchmarks, and a constructive gadget testing the joint usefulness of P^{EE} and χ .
6. To identify structural and task-dependent conditions under which the proposed hypergraph-derived quantities carry usable signal or become inert, while delineating the limitations of the construction and directions for future work.

Reproducibility statement. The code used to reproduce the experiments is available at <https://github.com/adriruiz12/bachelor-thesis-eompp>. Experimental settings and implementation details are described in Section 5.

IMPORTANT NOTE (post-submission): this repository (and this document) corresponds to the version submitted for the TFG. The code has since been further developed to extend the contribution beyond the scope evaluated here. The current, updated version can be found at: <https://github.com/adriruiz12/hypergraph-random-walk-encodings>

2. Graphs and Message Passing

2.1. Graphs, Features, and Permutation Symmetry

Definition 2.1 (Graph). *A (finite, simple) undirected graph is a pair $G = (V, E)$ where V is a finite set of vertices and $E \subseteq \{\{u, v\} : u, v \in V, u \neq v\}$ is a set of edges. A directed graph has $E \subseteq \{(u, v) \in V \times V : u \neq v\}$.*

In graph learning, nodes and/or edges carry attributes. An attributed graph can be represented as $G = (V, E, X)$, where $X \in \mathbb{R}^{n \times F}$ is the node-feature matrix, $n = |V|$, and F is the dimensionality of each node-feature vector; the v -th row of X , denoted $x_v \in \mathbb{R}^F$, encodes the features of node v (Wu et al., 2021). When node attributes are unavailable, it is common to set X to a constant (e.g. all-ones) to keep a uniform notation. A common alternative representation is (A, X) where $A \in \mathbb{R}^{n \times n}$ is the adjacency matrix (in the unweighted case, $A_{ij} \in \{0, 1\}$ indicates whether an edge is present; in weighted graphs, $A_{ij} \in \mathbb{R}_+$ stores an edge weight, and absent edges correspond to $A_{ij} = 0$). For undirected graphs, the adjacency matrix is symmetric ($A_{ij} = A_{ji}$). For directed graphs, A_{ij} represents an edge with source v_i and target v_j , so A need not be symmetric. Edge features can be stored as tensors $E \in \mathbb{R}^{n \times n \times K}$, or via typed relations $\{A^{(r)}\}_{r \in \mathcal{R}}$ in multi-relational graphs.

Example 2.2 (Molecular graphs). *A canonical instance of attributed graphs is molecular data: nodes are atoms with node-type features, while bonds are edges with bond-type features. One convenient encoding is an adjacency tensor $A \in \mathbb{R}^{n \times n \times Y}$ whose Y slices correspond to bond types, together with a node-type matrix $X \in \mathbb{R}^{n \times T}$ whose T columns correspond to atom types.*

Definition 2.3 (Neighborhoods and degrees). *For a node $v \in V$, its (1-hop) neighborhood is*

$$N(v) := \{u \in V : \{u, v\} \in E\} \quad (\text{or } N(v) := \{u \in V : (u, v) \in E\} \text{ in the directed case}),$$

and its degree is $\deg(v) := |N(v)|$. The k -hop neighborhood $N^k(v)$ is the set of nodes at graph distance at most k from v .

2.1.1. Linear algebra: degrees and Laplacians

We now recall the basic linear-algebraic operators that encode graph connectivity and that will later reappear as propagation operators in message passing.

Definition 2.4 (Degree matrix). *Let A be the (possibly weighted) adjacency matrix of an undirected graph on n nodes. The degree matrix is $D = \text{diag}(d_1, \dots, d_n)$, where*

$$d_i := \sum_j A_{ij}.$$

From the adjacency matrix A and the degree matrix D , one constructs the graph Laplacian, one of the central operators in graph theory. Intuitively, it compares the value of a node-wise signal at a node with the values at its neighbors, thereby measuring variation or lack of smoothness over the graph. This is why Laplacian-based operators naturally appear in diffusion processes, spectral methods, and later in message-passing architectures.

Definition 2.5 (Graph Laplacian). *Let G be an undirected, possibly weighted graph with adjacency matrix A and degree matrix D . The combinatorial (unnormalized) Laplacian is*

$$L := D - A.$$

For graphs with no isolated vertices, the random-walk Laplacian and the symmetric normalized Laplacian are defined, respectively, by

$$L_{\text{rw}} := I - D^{-1}A, \quad L_{\text{sym}} := I - D^{-1/2}AD^{-1/2}.$$

The combinatorial Laplacian compares a node-wise signal at a node with its values at the neighbors, encoding a notion of smoothness over the graph. The normalized versions reappear in graph learning as propagation rules or linear parts of message-passing layers (Kipf and Welling, 2017; Wu et al., 2021); their derivations and the similarity $L_{\text{sym}} = D^{1/2}L_{\text{rw}}D^{-1/2}$ are given in von Luxburg (2007).

2.1.2. Permutation symmetry

Unlike grid-structured data, graphs do not come with a canonical ordering of nodes: any relabeling of indices represents the same underlying object. Consequently, graph learning models should be compatible with node relabelings: graph-/set-level predictions should be invariant, while node-level predictions (e.g. embeddings) should be equivariant. This requirement is what ultimately forces aggregation operators (that is, a permutation-invariant function that combines the features or messages coming from a node’s neighbors into a single summary representation) to ignore the ordering of neighbors (Maron et al., 2019).

Throughout, let π denote a permutation of $\{1, \dots, n\}$, i.e. a relabeling of node indices.

Definition 2.6 (Permutation action on node-indexed objects). *A permutation π acts on node-indexed objects by reindexing*

$$\begin{aligned} X &\mapsto \pi(X) && (\text{permute rows}), \\ A &\mapsto \pi(A) && (\text{permute rows and columns}), \\ S &\mapsto \pi(S), \end{aligned}$$

where $A \in \mathbb{R}^{n \times n}$ is the adjacency matrix, $X \in \mathbb{R}^{n \times d}$ the node-feature matrix, and $S \subseteq V$ a target node set.

Definition 2.7 (Permutation invariance (graph-/set-level)). *A function producing a graph-/set-level output is permutation invariant if, and only if,*

$$f(S, A, X) = f(\pi(S), \pi(A), \pi(X)) \quad \forall \pi,$$

i.e., the output does not change under relabeling of the input.

Definition 2.8 (Permutation equivariance (node-level)). *A function producing node-level outputs (e.g. embeddings) is permutation equivariant if, and only if,*

$$\pi(f(A, X)) = f(\pi(A), \pi(X)) \quad \forall \pi,$$

i.e., outputs permute consistently with inputs.

Neighborhood information is inherently unordered (a multiset, see Definition 2.11), so any aggregation of neighbor messages must be permutation-invariant (e.g. sum/mean/max/attention pooling (Veličković et al., 2018)). This will be made explicit in the MPNN definition in the next section.

2.2. Message Passing Neural Networks

Message Passing Neural Networks (MPNNs) provide a common language for many GNN architectures. Their key design principle is to update node representations by exchanging information along edges, while respecting permutation symmetry (Section 2.1.2). We first recall the standard layer definition, and then reformulate it in an operator-centric language that will later extend naturally to higher-order structures.

Definition 2.9 (MPNN layer Gilmer et al., 2017). *Let $h_v^{(\ell)} \in \mathbb{R}^d$ denote the representation of node v at layer ℓ , and let e_{uv} denote (optional) edge features. An MPNN layer computes a message from neighbors and updates each node via*

$$m_v^{(\ell+1)} = \text{AGG}\left(\{\text{MLP}_e(h_v^{(\ell)}, h_u^{(\ell)}, e_{uv}) : u \in N(v)\}\right), \quad (1)$$

$$h_v^{(\ell+1)} = U_\ell(h_v^{(\ell)}, m_v^{(\ell+1)}), \quad (2)$$

where $N(v)$ is the neighborhood of v , AGG is a permutation-invariant aggregation map over multisets (e.g. sum, mean, max, attention pooling), and U_ℓ is a learnable update map.

In simpler words, an MPNN layer has three ingredients:

- A message function that computes information sent from neighbors.
- A permutation-invariant aggregation of incoming messages.
- An update map that combines the previous node state with the aggregated neighborhood information.

This local viewpoint will later generalize to higher-order domains by changing the carrier objects and the propagation operator.

Remark 2.10 (Operator-centric view). *Many popular architectures can be expressed in matrix form as*

$$H^{(\ell+1)} = \sigma(P_{\text{prop}} H^{(\ell)} W^{(\ell)}),$$

where $H^{(\ell)} \in \mathbb{R}^{n \times d}$ stacks node representations, P_{prop} is a (normalized) propagation operator, and $W^{(\ell)}$ is learnable. This global form emphasizes that a layer consists of two conceptually distinct parts: a learnable feature transformation ($W^{(\ell)}$) and a propagation rule (P_{prop}). Choosing P_{prop} is central when moving from graphs to higher-order structures such as hypergraphs and complexes.

Definition 2.11 (Multisets and invariant pooling). *A multiset is an unordered collection in which repetitions of elements are allowed. In message passing, neighborhood aggregation acts on multisets of incoming messages, so the aggregation rule must be invariant under reordering. Common choices are*

$$\sum_{u \in N(v)} m_{u \rightarrow v}^{(\ell)}, \quad \frac{1}{|N(v)|} \sum_{u \in N(v)} m_{u \rightarrow v}^{(\ell)}, \quad \max_{u \in N(v)} m_{u \rightarrow v}^{(\ell)}$$

(componentwise in the last case).

A standard template for permutation-invariant set functions is given by the Deep Sets representation (Zaheer et al., 2017):

$$q(S) = \phi\left(\sum_{a \in S} \psi(a)\right),$$

where ψ encodes individual elements, the summation aggregates them in a permutation-invariant way, and ϕ maps the aggregated summary to the final output. This explains why sum-based pooling, followed by a learnable map, is a natural design pattern in MPNNs.

Remark 2.12 (Equivalent MSG/AGG/UPT notation). *Equations (1) and (2) are often written by separating message computation, aggregation, and update. Initialize $h_v^{(0)} := x_v$. For $\ell = 1, \dots, L$:*

$$\begin{aligned} m_{u \rightarrow v}^{(\ell)} &= \text{MSG}^{(\ell)}(h_v^{(\ell-1)}, h_u^{(\ell-1)}, e_{uv}), & u \in N(v); \\ a_v^{(\ell)} &= \text{AGG}^{(\ell)}(\{m_{u \rightarrow v}^{(\ell)} : u \in N(v)\}); \\ h_v^{(\ell)} &= \text{UPT}^{(\ell)}(h_v^{(\ell-1)}, a_v^{(\ell)}). \end{aligned}$$

Typically, $\text{MSG}^{(\ell)}$ and $\text{UPT}^{(\ell)}$ are parameterized (e.g. by MLPs), whereas $\text{AGG}^{(\ell)}$ must be permutation-invariant.

A typical message passing layer scales approximately as $\mathcal{O}(|E|d)$, where d is the embedding dimension, since each edge contributes to message computation at most once per layer. It also encodes a specific inductive bias: after L layers, $h_v^{(L)}$ can depend only on nodes in the L -hop neighborhood of v . This locality is both a strength (e.g. scalability on sparse graphs) and a limitation (e.g. oversquashing of long-range dependencies; Definition 3.12).

2.3. Canonical Examples: GCN and GIN

Before moving to higher-order domains, it is useful to see how a canonical GNN architecture fits into the MPNN template of Section 2.2, where normalized linear propagation appears as one particular choice of propagation operator. This perspective will be important later, when the main design question becomes how to choose the propagation operator in higher-order structures.

Example 2.13 (GCN). *Graph Convolutional Networks employ normalized linear aggregation, typically using $\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$ with self-loops, to stabilize training and account for varying degrees. A GCN layer (Kipf and Welling, 2017) (with self-loops) is*

$$H^{(\ell)} = \sigma\left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(\ell-1)} W^{(\ell)}\right), \quad \tilde{A} = A + I,$$

which corresponds to a linear message, normalized sum aggregation, and pointwise nonlinearity. The propagation matrix is the symmetric normalized adjacency $\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} = I - \tilde{L}_{\text{sym}}$, where $\tilde{L}_{\text{sym}}$ is the symmetric normalized Laplacian (von Luxburg, 2007) of the self-looped graph $(\tilde{A}, \tilde{D})$.

Definition 2.14 (1-WL / color refinement). *Let $G = (V, E)$ be a graph with an initial node labeling $\{c_v^{(0)}\}_{v \in V}$. The 1-dimensional Weisfeiler-Leman procedure iteratively refines node colors by*

$$c_v^{(t+1)} = \text{HASH}\left(c_v^{(t)}, \{\!\!\{c_u^{(t)} : u \in N(v)\}\!\!\}\right),$$

where $\{\!\!\{\cdot\}\!\!\}$ denotes a multiset and HASH is injective. Two nodes receive the same new color if and only if they had the same previous color and the same multiset of neighbor colors.

Two graphs are 1-WL-indistinguishable if, after stabilization, their multisets of node colors coincide at every iteration. The 1-WL test is an efficient graph isomorphism heuristic but not a complete algorithm.

Remark 2.15. *The relevance of 1-WL in graph learning is that standard message passing architectures also update each node by combining its current state with a permutation-invariant aggregation of neighbor states. As a result, for standard permutation-invariant neighborhood-aggregation MPNNs, their ability to distinguish non-isomorphic graphs is upper-bounded by 1-WL. Conversely, with sufficiently expressive injective aggregation and update maps, one can match the power of 1-WL (Xu et al., 2019).*

Example 2.16 (GIN). *A canonical neural architecture achieving this bound is the Graph Isomorphism Network, whose layer has the form*

$$h_v^{(\ell+1)} = \text{MLP}^{(\ell)} \left((1 + \varepsilon^{(\ell)}) h_v^{(\ell)} + \sum_{u \in N(v)} h_u^{(\ell)} \right).$$

With sufficiently expressive multilayer perceptrons and injective multiset aggregation, GIN matches the discriminative power of 1-WL. This makes 1-WL the natural expressivity baseline for ordinary MPNNs (Xu et al., 2019).

Remark 2.17. *The 1-WL test is only the first level of a hierarchy. For $k \geq 2$, the k -dimensional Weisfeiler-Leman test (k -WL) assigns colors to ordered k -tuples of vertices rather than to individual vertices, refining them through neighborhoods obtained by replacing one coordinate at a time. The 1-WL and 2-WL tests have the same distinguishing power, whereas $(k+1)$ -WL is strictly stronger than k -WL for every $k \geq 2$ (Besta et al., 2026). This hierarchy is one of the two senses in which message passing can be made “higher-order”: here the underlying domain remains an ordinary graph, but hidden states are lifted from nodes to k -tuples of nodes. This contrasts with the route taken in the rest of this thesis, where the carrier remains vertex-centered but the underlying domain itself becomes higher-order (a hypergraph). The two notions are largely orthogonal, and we return to the relationship between them when interpreting the synthetic experiment of Section 5.1.*

3. Higher-Order Structures and Propagation Operators

We now move from pairwise graph representations to higher-order domains and to the propagation operators naturally associated with them.

3.1. Why Higher-Order Structures?

Graph learning studies data where entities are coupled by relations. Graphs encode only dyadic ones, which is a real limitation in domains where the natural relations are polyadic (groups) or multi-rank (nodes-edges-faces-cells): distinct higher-order configurations can project to the same pairwise graph, causing ordinary message passing to treat them identically. As a canonical example, “three authors writing one paper together” and “three separate pairwise collaborations” both project to the same triangle graph, but are different polyadic patterns. This motivates higher-order graph data models (HOGDMs) and higher-order graph neural networks (HOGNNs) (Hajij et al., 2023; Besta et al., 2026).

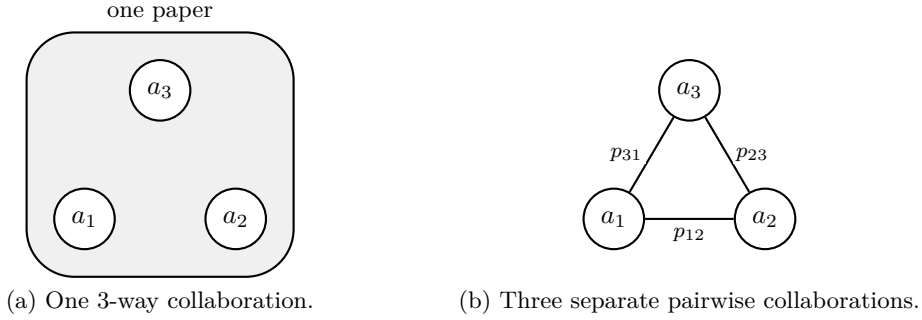


Figure 1: two different collaboration patterns on the same three authors. Left: a single polyadic interaction (one joint paper). Right: three distinct dyadic interactions. Both may project to the same triangle graph, but they encode different higher-order structure.

A useful organizing axis is whether higher-order interactions are set-type (arbitrary subsets, no hierarchy) or hierarchical (closed under faces/incidence). For instance, hypergraphs represent set-type relations; simplicial and cell complexes represent hierarchical relations; and combinatorial complexes aim to unify both (Hajij et al., 2023; Besta et al., 2026).

3.2. Hypergraphs

Definition 3.1 (Hypergraph). *A (finite) hypergraph is $H = (V, \mathcal{E})$, where V is a finite vertex set and $\mathcal{E} = (e_\alpha)_{\alpha \in A}$ is a finite family of non-empty hyperedges $e_\alpha \in \mathcal{P}(V) \setminus \{\emptyset\}$ (subsets of arbitrary cardinality). We allow $\mathcal{E}$ to be a multiset: the same subset of V may occur as several hyperedges, with its multiplicity recorded. This matters for real incidence data (e.g. co-citation hypergraphs) where one vertex set can be induced by several distinct relations.*

Definition 3.2 (Incidence matrix). *Let $n = |V|$ and $m = |\mathcal{E}|$. The incidence matrix $B \in \{0, 1\}^{n \times m}$ is defined by $B(v, e) = 1$ if, and only if, $v \in e$. Hyperedge-weighted versions incorporate hyperedge weights, leading to generalized degree matrices and Laplacians that drive hypergraph diffusions (Carletti et al., 2020).*

Definition 3.3 (Clique expansion). *Given a hypergraph $H = (V, \mathcal{E})$, its clique expansion is the graph obtained by replacing each hyperedge $e \in \mathcal{E}$ by the complete graph on the vertex set e , i.e., by adding all pairwise edges among vertices in e .*

Clique expansion is lossy, as it replaces each hyperedge by all pairwise edges among its nodes, collapsing distinct polyadic patterns. Many hypergraph message passing methods therefore operate directly on incidence or bipartite representations (Besta et al., 2026; Hajij et al., 2023).

Definition 3.4 (Bipartite graph). *A graph $G = (U \sqcup W, F)$ is bipartite if its vertex set can be partitioned into two disjoint classes U and W such that every edge has one endpoint in U and the other in W .*

Definition 3.5 (Bipartite expansion). *Given a hypergraph $H = (V, \mathcal{E})$, its bipartite expansion is the bipartite graph with vertex classes V and $\mathcal{E}$, where $v \in V$ and $e \in \mathcal{E}$ are adjacent if, and only if, $v \in e$. Equivalently, this incidence relation is encoded by the incidence matrix B . If $\mathcal{E}$ contains repeated hyperedges, each occurrence yields a distinct vertex on the $\mathcal{E}$ side, consistently with the columns of B .*

Example 3.6. *These definitions (3.1-3.5) can be better visualized in Figure 2.*

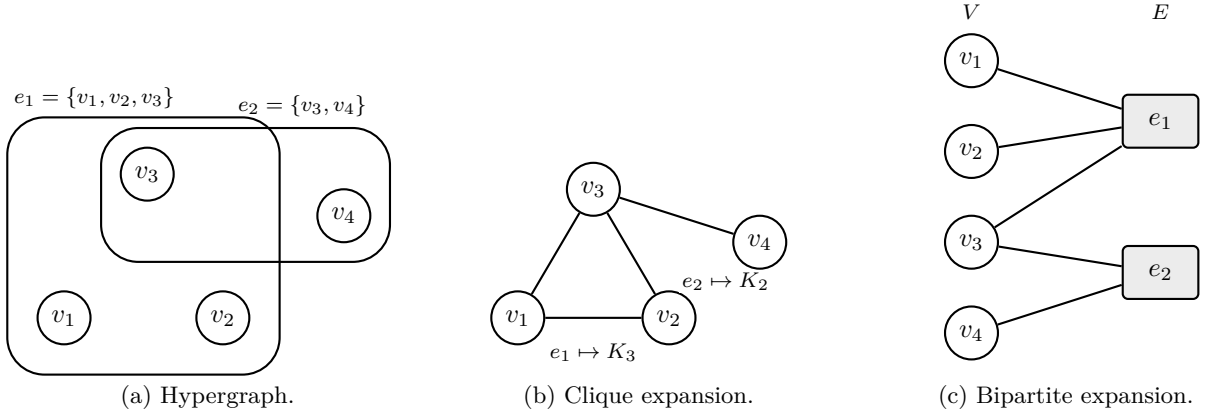


Figure 2: a hypergraph and two standard graphifications. Left: the original hypergraph with hyperedges $e_1 = \{v_1, v_2, v_3\}$ and $e_2 = \{v_3, v_4\}$; middle: the clique expansion, where each hyperedge is replaced by a complete graph on its incident vertices; right: the bipartite expansion, where vertices and hyperedges become two node classes connected by incidence edges.

3.3. Beyond Hypergraphs: Simplicial, Cellular, and Combinatorial Complexes

Definition 3.7 (Simplicial complex). *Let V be a non-empty set. A simplicial complex K is a collection of non-empty subsets of V such that:*

- All singletons $\{v\}$ belong to K .
- If $\sigma \in K$ and $\emptyset \neq \tau \subseteq \sigma$, then $\tau \in K$.

A set σ with $|\sigma| = d + 1$ is a d -simplex. Thus, a 0-simplex is a vertex, a 1-simplex an edge, a 2-simplex a filled triangle, and a 3-simplex a filled tetrahedron. If $\tau \subset \sigma$, then τ is called a face of σ , and σ is called a coface of τ .

Simplicial complexes encode higher-order interactions and all lower-dimensional faces. This enables canonical boundary operators and rank-wise Laplacians (Hodge Laplacians), which are

central in topological signal processing and simplicial message passing, but are not developed further in this thesis (Hajij et al., 2023).

Definition 3.8 (Cellular complex). *A cellular (CW) complex is a space built from cells of different dimensions (vertices, edges, polygons, and higher-dimensional cells) that are attached to lower-dimensional cells via attaching maps. Unlike simplicial complexes, the cells need not be simplices, which makes cellular complexes suitable for polygonal or more general non-simplicial geometries.*

In learning, cellular message passing often mirrors simplicial message passing, but uses cell-specific incidence structures (Hajij et al., 2023). We do not develop this further here.

A key idea in topological deep learning is to specify local interactions through neighborhood functions rather than a single adjacency. This supports multiple notions of locality (incidence-like, adjacency-like, path-based, etc.) within the same domain (Hajij et al., 2023).

Definition 3.9 (Combinatorial complex (Hajij et al., 2023; 2025)). *A combinatorial complex is a triple (S, X, rk) where S is a finite vertex set, $X \subseteq \mathcal{P}(S) \setminus \emptyset$ is a set of cells containing every singleton s , and $\text{rk} : X \rightarrow \mathbb{Z}_{\geq 0}$ is an order-preserving rank function satisfying $\text{rk}(s) = 0$ for every $s \in S$, i.e. $x \subseteq y \implies \text{rk}(x) \leq \text{rk}(y)$ for $x, y \in X$. Neighborhood functions define relations between cells, giving rise to one or more propagation operators.*

3.4. Message Passing as an Operator Choice

Higher-order message passing goes beyond node-only state representations. Depending on the domain, states can live on:

- Nodes only (graph-style).
- Nodes and hyperedges (incidence-based hypergraph networks).
- Simplices/cells at multiple ranks (complex-based networks).
- Ranked cells in a combinatorial complex (CCNN¹/CTNN² style).

Thus, the main modeling choice is not only how to propagate information, but also which objects are allowed to carry hidden states.

Let $\mathcal{X}$ denote the carrier set, i.e., the objects that carry embeddings. A generic higher-order message passing layer takes the form

$$\begin{aligned} m_x^{(\ell+1)} &= \text{AGG}_{y \in \mathcal{N}(x)} \text{MSG}_\ell \left(h_x^{(\ell)}, h_y^{(\ell)}, \text{rel}(x, y) \right), \\ h_x^{(\ell+1)} &= U_\ell \left(h_x^{(\ell)}, m_x^{(\ell+1)} \right), \end{aligned}$$

where $\mathcal{N}(x)$ is a neighborhood of x in $\mathcal{X}$, and $\text{rel}(x, y)$ encodes the relevant higher-order relation between x and y (e.g., incidence type, shared faces/cofaces, or hyperedge metadata). This template subsumes many higher-order architectures once $\mathcal{X}$ and $\mathcal{N}$ are specified appropriately (Hajij et al., 2023; Besta et al., 2026).

For higher-order architectures with a linear propagation stage, the same algebraic form $H^{(\ell+1)} = \sigma(P_{\text{prop}} H^{(\ell)} W^{(\ell)})$ from Remark 2.10 applies, with $H^{(\ell)}$ now stacking the states of the chosen

¹Combinatorial Complex Neural Networks (Hajij et al., 2023).

²Copresheaf Topological Neural Networks (Hajij et al., 2025).

carrier objects and P_{prop} encoding propagation between them. Depending on the construction, P_{prop} may be:

- A one-step random walk kernel.
- A lazy or biased walk.
- A restart kernel.
- A multi-step diffusion.
- A higher-order lift of any of the above.

Thus, choosing a random-walk kernel amounts to choosing the linear propagation rule that determines which representations are mixed, and with what weights, before the nonlinear activation. This operator viewpoint is explicit in graph MPNNs and remains valid, in a more structured form, on higher-order domains (Gilmer et al., 2017; Hajij et al., 2023; Besta et al., 2026).

A standard hypergraph-native paradigm alternates aggregation between nodes and hyperedges:

$$\begin{aligned} h_e^{(\ell+1)} &= \phi_e\left(h_e^{(\ell)}, \text{AGG}_{v \in e} \psi_{v \rightarrow e}(h_v^{(\ell)})\right), \\ h_v^{(\ell+1)} &= \phi_v\left(h_v^{(\ell)}, \text{AGG}_{e \ni v} \psi_{e \rightarrow v}(h_e^{(\ell+1)})\right), \end{aligned}$$

with permutation-invariant aggregation inside hyperedges. This mirrors node-hyperedge-node random walks used to define hypergraph diffusion operators (Carletti et al., 2020).

The node-hyperedge-node scheme can be interpreted as a local realization of a hypergraph propagation operator; diffusion-inspired layers make this operator explicit.

Carletti et al. (2020) derives a generalized random walk Laplace operator driven by hyperedge-mediated exchanges, reducing to the classical random-walk Laplacian when all hyperedges have size two. Its associated transition kernel can serve as a principled propagation matrix P for hypergraph message passing.

Beyond the choice of a particular hypergraph Laplacian, diffusion-inspired hypergraph layers involve several modeling decisions, including how to sample incident hyperedges and how to choose destinations within them (Carletti et al., 2020; Coupette et al., 2023). Once a one-step kernel is fixed, restart and multi-step kernels can be formed as in Section 3.5. These choices modify the effective propagation operator P and the local patterns it distinguishes (Eidi and Otter, 2025).

3.5. Random walk with restart and PageRank

In this section, the base matrix P is assumed to be row-stochastic, meaning that $P_{ij} \geq 0$ for all i, j and $\sum_{j=1}^n P_{ij} = 1$ for every row i , so that P defines a probability transition kernel.

Definition 3.10 (Random walk with restart). *Given a base transition matrix P (e.g. the simple random walk $P = D^{-1}A = I - L_{\text{rw}}$), a restart parameter $\gamma \in (0, 1)$, and a restart distribution r , define*

$$P_{\text{restart}} = (1 - \gamma)P + \gamma \mathbf{1} r^{\top}.$$

Thus, at each step the walker either follows the graph with probability $1 - \gamma$ or teleports according to r with probability γ .

A closely related object is personalized PageRank (Jeh and Widom, 2003), which is the canonical geometric multi-step extension of a random walk with restart: it aggregates walks of all lengths, but discounts longer trajectories exponentially, thereby balancing global diffusion with retention of source locality:

$$K_{\text{ppr}}(\alpha) = (1 - \alpha) \sum_{k \geq 0} \alpha^k P^k = (1 - \alpha)(I - \alpha P)^{-1}, \quad 0 < \alpha < 1,$$

whenever the inverse is well-defined.

Remark 3.11. *Restart changes the propagation geometry in two ways. First, it continually reinjects probability mass according to the restart distribution, thereby limiting the influence of long trajectories. Second, it interpolates between local diffusion and global accessibility. For message passing, this is particularly attractive because it gives a principled way to balance locality and globality. This same tension between local aggregation and longer-range communication is central in oversquashing discussions (Alon and Yahav, 2021; Taha et al., 2025).*

Definition 3.12 (Oversquashing). *Oversquashing is the phenomenon by which exponentially many long-range messages are compressed through narrow neighborhood bottlenecks into fixed-size representations, limiting information transport across the graph (Alon and Yahav, 2021; Taha et al., 2025).*

3.6. Random Walks and Diffusion on Hypergraphs

Having developed restart and PageRank for ordinary graphs, we now turn to walks that are intrinsically hypergraph-native.

The most natural hypergraph walk is an alternating two-stage process:

1. From the current node v , choose an incident hyperedge $e \ni v$.
2. Inside e , choose a destination node $u \in e$.

This perspective is already implicit in microscopic hypergraph diffusion models and is particularly natural from the incidence viewpoint used in hypergraph message passing (Carletti et al., 2020; Hajij et al., 2023).

Definition 3.13 (General node-hyperedge-node kernel). *Let $H = (V, \mathcal{E})$ be a hypergraph. For each node v , let*

$$\mathcal{E}(v) := \{e \in \mathcal{E} : v \in e\}.$$

For every v with $\mathcal{E}(v) \neq \emptyset$, choose:

- *A hyperedge-selection law $\alpha_v(e)$ on $\mathcal{E}(v)$.*
- *A within-hyperedge destination law $\beta_{e,v}(u)$ on e .*

Then the induced node-level transition kernel is

$$P_{vu} = \sum_{e \in \mathcal{E}(v)} \alpha_v(e) \beta_{e,v}(u).$$

A first lift is not genuinely hypergraph-native: project the hypergraph to a weighted graph, then run an ordinary graph walk.

Definition 3.14 (Projected graph). *Given a hypergraph $H = (V, \mathcal{E})$, its projected weighted graph has the same vertex set V and weighted adjacency*

$$W_{uv} = \sum_{\substack{e \in \mathcal{E} \\ u, v \in e, u \neq v}} \omega_e$$

for some hyperedge weights $\omega_e \geq 0$.

Unlike the unweighted clique expansion of Section 3.2, which only records whether u and v share some hyperedge, the projected graph can retain weighted summaries of multiplicity and cardinality through the choice of ω_e . It does not, however, retain this information exactly in general, since different multisets of shared hyperedges may induce the same scalar weight W_{uv} . Setting $\omega_e = 1$ for all e and thresholding $W_{uv} > 0$ recovers the unweighted clique expansion as a special case. For every vertex v with positive weighted degree, the corresponding projected transition probabilities are

$$P_{vu}^{\text{proj}} = \frac{W_{vu}}{\sum_{w \in V} W_{vw}}.$$

Equivalently, after restricting to positive-degree rows, $P^{\text{proj}} = D_W^{-1}W$, where $D_W := \text{diag}(W\mathbf{1})$. A separate convention is required for zero-degree rows.

Remark 3.15. *The projected walk is computationally simple and reuses ordinary graph theory, but it collapses polyadic structure into pairwise couplings. Distinct hypergraphs may induce the same projection. This trade-off between reuse of graph machinery and loss of higher-order information is a recurrent theme in hypergraph learning (Hajij et al., 2023; Besta et al., 2026).*

A more intrinsic choice is: from the current node, choose one incident hyperedge uniformly; then choose another node inside that hyperedge uniformly.

Definition 3.16 (Uniform-incidence higher-order walk). *In this definition, $\mathcal{E}(v)$ is restricted to the hyperedges incident to v with cardinality at least two, and $d_H(v) := |\mathcal{E}(v)|$. Assume $d_H(v) > 0$ and adopt the non-lazy convention (the walker cannot remain at its current vertex, i.e. $\beta_{e,v}(v) = 0$). Define*

$$\alpha_v(e) = \frac{1}{d_H(v)} \quad \text{for } e \in \mathcal{E}(v),$$

and

$$\beta_{e,v}(u) = \begin{cases} \frac{1}{|e| - 1}, & u \in e \setminus \{v\}, \\ 0, & u = v, \end{cases}$$

because of the non-lazy convention. Then, by Definition 3.13

$$P_{vu}^{\text{HO}} = \begin{cases} \sum_{e \ni v, u} \frac{1}{d_H(v)} \cdot \frac{1}{|e| - 1}, & u \neq v, \\ 0, & u = v. \end{cases}$$

This walk treats each incident hyperedge equally from the current node's perspective. Because the within-hyperedge normalization uses $|e| - 1$, transitions through large hyperedges are diluted across many possible destinations. This is precisely the equal-edges random walk of Coupette

et al. (2023), closely related to earlier two-step hypergraph random walks such as that of Zhou et al. (2006).

Another natural family favors large hyperedges at the first stage.

Definition 3.17 (Size-biased hypergraph walk). *As in Definition 3.13, this is obtained by specifying the hyperedge-selection law and the within-hyperedge destination law explicitly. Fix a nonnegative function $g : \mathbb{N}_{\geq 2} \rightarrow \mathbb{R}_{\geq 0}$ such that $\sum_{f \in \mathcal{E}(v)} g(|f|) > 0$ for every v with $d_H(v) > 0$. Define*

$$\alpha_v^{(g)}(e) = \frac{g(|e|)}{\sum_{f \in \mathcal{E}(v)} g(|f|)},$$

and, as in Definition 3.16, keep the same within-hyperedge destination rule

$$\beta_{e,v}(u) = \begin{cases} \frac{1}{|e| - 1}, & u \in e \setminus \{v\}, \\ 0, & u = v. \end{cases}$$

Then

$$P_{vu}^{(g)} = \begin{cases} \sum_{e \ni v, u} \frac{g(|e|)}{\sum_{f \in \mathcal{E}(v)} g(|f|)} \cdot \frac{1}{|e| - 1}, & u \neq v, \\ 0, & u = v. \end{cases}$$

Special cases include:

- $g(k) \equiv 1$: uniform-incidence walk.
- $g(k) = k$ or $g(k) = k - 1$: size-biased walk, in the spirit of Carletti et al. (2020), who select incident hyperedges with probability proportional to their size.
- $g(k) = k^s$: nonlinear size bias.

Remark 3.18. *This family makes explicit a genuine modeling choice: should large hyperedges be more likely to be entered, or should every incident hyperedge count equally from the point of view of the current node? That choice is not only probabilistic, but it also changes what the model will regard as locally important in a message-passing context.*

After restricting P_H to the non-isolated vertices, where its rows are stochastic, the restart and personalized PageRank constructions of Section 3.5 carry over by direct substitution:

$$P_{H,\text{restart}} = (1 - \gamma)P_H + \gamma \mathbf{1} r^\top, \quad K_{H,\text{ppr}}(\eta) = (1 - \eta) \sum_{k \geq 0} \eta^k P_H^k = (1 - \eta)(I - \eta P_H)^{-1},$$

where r is a restart distribution on vertices.

Remark 3.19. *As in the graph case, $K_{H,\text{ppr}}(\eta)$ sees more than P_H alone: a single step only reaches the immediate, one-hop neighborhood induced by the chosen hyperedge-selection rule, whereas mixing in longer walks lets farther-away parts of the hypergraph contribute too, with weight decaying geometrically in η . Nearby structure still dominates, but is no longer the whole picture. Small η keeps the walk local, while η close to 1 lets it explore much further into the hypergraph.*

4. The EO-Pattern MPNN

4.1. Motivation: What Clique Expansions Forget

As established in Sections 3.1 and 3.2, the clique expansion is computationally convenient (it turns a hypergraph into an ordinary graph, so that standard graph architectures apply unmodified) but lossy: it records whether two vertices co-occur in some hyperedge, but forgets which hyperedge generated that adjacency. Consequently, the interaction between two adjacent vertices is determined only by the fact of adjacency, never by whether it came from a small hyperedge, a large one, or several hyperedges of different cardinalities. Yet this distinction may matter: a pair of vertices co-occurring inside a group of size two need not carry the same structural significance as a pair co-occurring inside a group of size twenty, and an unweighted clique-based GNN cannot tell the two cases apart.

One response is to abandon purely vertex-centered message passing and operate directly on node-hyperedge incidence, as in the alternating node-to-hyperedge and hyperedge-to-node aggregation of Section 3.4 (e.g. Chien et al. (2022)’s AllSet). The aim of the construction developed in this section is different. We keep the vertex-centered MPNN form: messages are still exchanged between vertices, and the aggregation support is still the pairwise neighborhood induced by the hypergraph. What changes is the information attached to each pairwise interaction.

The guiding idea is therefore to distinguish between the support of message passing and the structure used to define the messages. The support may be pairwise, as in a clique-expanded graph, but the weights and descriptors used by the message function can still be computed from the original hypergraph incidence data. In this way, the layer remains close to a standard edge-conditioned MPNN (Gilmer et al., 2017), while exposing to the model selected higher-order information that the unweighted clique expansion would erase.

The two quantities used below follow this principle. The equal-edges propagation kernel P^{EE} weights a transition from a vertex v to a vertex u by summing over the hyperedges through which v can reach u , with a normalization depending on the cardinality of each hyperedge. The descriptor χ_{vu} complements this scalar weight by recording a cardinality-sensitive summary of the hyperedges jointly incident to v and u . Together, they do not reconstruct the full hypergraph, nor are they meant to define a complete invariant of higher-order incidence. Rather, they provide a minimal way of enriching pairwise message passing with hypergraph-derived information.

This is the sense in which the proposed layer is designed to test what clique expansions forget. If two hypergraphs have the same clique expansion but differ in the cardinalities of the hyperedges that generate their pairwise adjacencies, then a standard clique-based GNN receives the same structural input in both cases. By contrast, a message function conditioned on P^{EE} and χ_{vu} can, in principle, react differently to those two hypergraphs. The rest of this section formalizes this idea, which is tested directly in the synthetic experiment of Section 5.1.

4.2. EO-Inspired Cardinality-Pattern Message Passing

The aim of this section is to propose a simple vertex-centered, hypergraph-derived message passing layer inspired by the constructions of Eidi and Otter (2025). The guiding principle is that, even when aggregation is ultimately performed over pairwise neighbors, the messages

should not depend only on the induced adjacency, but also on a cardinality-sensitive summary of the hyperedges through which two vertices interact.

Concretely, we want a vertex-centered MPNN whose propagation operator is the equal-edges random walk of Coupette et al. (2023), the same kernel underlying the equal-edges Ollivier-Ricci curvature of Eidi and Otter (2025). We augment each message with a structural descriptor encoding the cardinalities of the hyperedges through which the source and target vertices are connected.

This contrasts with the relational-structure viewpoint of Taha et al. (2025), where topological message passing is formulated through a finite set of entities, relations, shift operators, and an associated influence graph. That framework is useful as an axiomatic and analytical language, especially for studying oversquashing (Definition 3.12) in topological message passing. However, the construction proposed here does not start from a relational graph or an influence graph. Instead, the message passing rule is defined directly on the hypergraph by using hyperedge incidence and cardinality-sensitive connectivity.

4.2.1. Hypergraph notation

Most of the notation below has already appeared, at least informally, in Section 3. Here we fix it precisely, specialize it to the loop-free setting used below, and introduce the few additional objects needed to define P^{EE} and χ_{vu} .

Recall the hypergraph $H = (V, \mathcal{E})$ of Definition 3.1, with $\mathcal{E}$ possibly a multiset. For a hyperedge $e \in \mathcal{E}$, its cardinality is denoted by $|e|$. In the implementation, repeated hyperedges are kept as distinct columns of the incidence matrix, whereas each individual hyperedge is treated as a set of distinct vertices.

For a vertex $v \in V$, define the set of incident hyperedges of size at least two by

$$\mathcal{E}(v) := \{e \in \mathcal{E} : v \in e, |e| \geq 2\},$$

refining the notation of Definition 3.13 by excluding loops, consistent with the non-lazy convention of Definition 3.16. The hypergraph degree of v is

$$d_H(v) := |\mathcal{E}(v)|.$$

For two distinct vertices $v, u \in V$, define the set of hyperedges jointly incident to v and u by

$$\mathcal{E}(v, u) := \{e \in \mathcal{E} : v \in e, u \in e\}.$$

The hypergraph neighborhood of v is then

$$N_H(v) := \{u \in V \setminus \{v\} : \mathcal{E}(v, u) \neq \emptyset\}.$$

Thus, $u \in N_H(v)$ whenever u and v co-occur in at least one hyperedge. However, unlike a clique expansion, we do not forget which hyperedges generate this adjacency. The set $\mathcal{E}(v, u)$ remains part of the message passing rule.

Remark 4.1 (Multiplicities). *The collections $\mathcal{E}(v)$ and $\mathcal{E}(v, u)$ are sub-multisets of $\mathcal{E}$: a subset occurring several times in $\mathcal{E}$ contributes that many elements. Consequently, $d_H(v)$ counts incident hyperedges with multiplicity, and every sum $\sum_{e \in \mathcal{E}(v, u)}$ in the sequel (in Z_{vu} , P^{EE} and χ_{vu}) runs over the family with multiplicity. When $\mathcal{E}$ has no repeated hyperedges, these collections reduce to ordinary sets and all definitions are unchanged.*

4.2.2. The equal-edges propagation kernel

Eidi and Otter (2025) characterize structural and regular equivalences in hypergraphs, distinguishing weak notions (which record only shared neighbors) from stronger notions (which also record the cardinalities of the shared hyperedges). This distinction motivates the two ingredients of the layer proposed here.

The propagation weight used below instantiates the uniform-incidence walk of Definition 3.16, which follows the equal-edges random walk formulation of Coupette et al. (2023), closely related to earlier two-step hypergraph random walks such as that of Zhou et al. (2006), but using the non-lazy convention in which the destination is chosen from $e \setminus \{v\}$, leading to the factor $|e| - 1$. We follow the equal-edges convention because it underlies the equal-edges Ollivier-Ricci curvature (EE-ORC) in Eidi and Otter (2025), the curvature used in their necessary condition for strong structural equivalence, which motivates the curvature-gate extension discussed in Section 6.

This random walk defines the probability of moving from a vertex v to another vertex u in one step. Starting at v , the walker first chooses uniformly one hyperedge incident to v . Since there are $d_H(v)$ such hyperedges, each one is chosen with probability $1/d_H(v)$. If the chosen hyperedge is e , the walker then chooses uniformly one of the $|e| - 1$ vertices of e different from v . Therefore, each hyperedge e containing both v and u contributes a term

$$\frac{1}{d_H(v)} \frac{1}{|e| - 1}$$

to the transition probability from v to u . Summing over all hyperedges containing both vertices gives

$$P^{\text{EE}}(v, u) = \frac{1}{d_H(v)} \sum_{e \in \mathcal{E}(v, u)} \frac{1}{|e| - 1}, \quad u \neq v,$$

whenever $d_H(v) > 0$. If $d_H(v) = 0$ ³, the neighborhood $N_H(v)$ is empty and the aggregated message $m_v^{(\ell+1)}$ is zero. In the implementation, the EO-Pattern layer therefore applies the node-wise update $h_v^{(\ell+1)} = \phi^{(\ell)}(h_v^{(\ell)}, \mathbf{0})$; no self-loop is inserted. In the node-classification experiments, the outer residual update subsequently adds the previous representation $h_v^{(\ell)}$.

Equivalently, choosing $\omega_e = 1/(|e| - 1)$ for every hyperedge with $|e| \geq 2$ in the projected-graph construction introduced above gives $W_{vu} = Z_{vu}$ and $D_W(v, v) = d_H(v)$, so that $P^{\text{proj}}(v, \cdot) = P^{\text{EE}}(v, \cdot)$ for every v with $d_H(v) > 0$. Thus, the kernel is hypergraph-derived in its construction while admitting an equivalent cardinality-weighted pairwise representation.

Proposition 4.2 (Row-stochasticity of the equal-edges kernel). *Let $v \in V$ be such that $d_H(v) > 0$. Then the equal-edges transition weights define a probability distribution over $N_H(v)$, that is,*

$$\sum_{u \in N_H(v)} P^{\text{EE}}(v, u) = 1.$$

Proof. By definition,

$$P^{\text{EE}}(v, u) = \frac{1}{d_H(v)} \sum_{e \in \mathcal{E}(v, u)} \frac{1}{|e| - 1}.$$

³If $d_H(v) = 0$, then v is isolated from the point of view of the equal-edges walk: it is not incident to any hyperedge of size at least two.

Non-negativity is immediate, since $d_H(v) > 0$ and $|e| - 1 > 0$ for every $e \in \mathcal{E}(v, u)$. It remains to verify that the weights sum to one. Therefore,

$$\begin{aligned} \sum_{u \in N_H(v)} P^{\text{EE}}(v, u) &= \frac{1}{d_H(v)} \sum_{u \in N_H(v)} \sum_{e \in \mathcal{E}(v, u)} \frac{1}{|e| - 1} \\ &= \frac{1}{d_H(v)} \sum_{e \in \mathcal{E}(v)} \sum_{u \in e \setminus \{v\}} \frac{1}{|e| - 1} \\ &= \frac{1}{d_H(v)} \sum_{e \in \mathcal{E}(v)} 1 = 1. \end{aligned}$$

□

Remark 4.3. *The factor $1/(|e| - 1)$ is essential. It means that a neighbor reached through a small hyperedge receives more weight than a neighbor reached through a very large hyperedge. Therefore, P^{EE} is not simply the random walk on the clique expansion: it is a cardinality-sensitive hypergraph propagation operator.*

4.2.3. Cardinality-pattern summary

The transition kernel $P^{\text{EE}}(v, u)$ determines how much weight vertex v assigns to the neighbor u during aggregation. However, this scalar weight does not fully describe the way in which v and u are connected. As emphasized by Eidi and Otter (2025), hypergraph connectivity can be considered at different levels of resolution: weak notions remember only the existence of shared neighbors, whereas stronger notions also take into account the cardinalities of the hyperedges through which those neighbors are reached.

Motivated by this distinction, we augment the equal-edges transition weight with a fixed, structural cardinality summary: as already noted in Section 4.1, the cases of a small hyperedge, a large hyperedge, or several hyperedges of different cardinalities should not necessarily produce the same type of message, even when their aggregated equal-edges weights are similar.

To define this summary, fix an integer $K \geq 2$. We use $K - 1$ cardinality bins: the first bins represent hyperedge sizes $2, 3, \dots, K - 1$, while the last bin pools together all sizes at least K . For each cardinality $r \geq 2$, let $\mathbf{c}_r \in \{0, 1\}^{K-1}$ be the corresponding one-hot cardinality encoding. Its nonzero entry is placed at position

$$b(r) = \min(r, K) - 1,$$

where positions are indexed from 1. This encoding is fixed and parameter-free.

For each pair v, u with $\mathcal{E}(v, u) \neq \emptyset$, define the normalizing factor

$$Z_{vu} := \sum_{e \in \mathcal{E}(v, u)} \frac{1}{|e| - 1}.$$

We then define the cardinality-pattern descriptor

$$\chi_{vu} := \frac{1}{Z_{vu}} \sum_{e \in \mathcal{E}(v, u)} \frac{1}{|e| - 1} \mathbf{c}_{|e|}.$$

Thus, χ_{vu} is a fixed, $1/(|e| - 1)$ -weighted histogram over the cardinality bins of the hyperedges shared by v and u . It carries no learnable parameters of its own: it is a deterministic function of the hypergraph structure. All learning is delegated to the message function ψ , which receives χ_{vu} as a structural input and maps it to a useful representation.

The role of χ_{vu} is therefore to refine pairwise adjacency with cardinality information from the original hypergraph. It does not encode the full multiset of shared hyperedges, and different local incidence patterns may lead to the same descriptor. Rather, it provides a cardinality-sensitive summary that allows the message function to distinguish some hypergraphs that would be identical after an unweighted clique expansion.

Remark 4.4 (Relation to the equivalence hierarchy of Eidi and Otter (2025)). *The normalizer Z_{vu} is the local weighted-neighborhood quantity appearing in their hierarchy, and χ_{vu} is inspired by their multiset structural equivalence notion: an intermediate level in their hierarchy, strictly between the EE-measure equivalence already captured by P^{EE} alone and full strong structural equivalence. They introduce this notion but do not operationalize it, explicitly leaving “a systematic study for future work”. χ_{vu} turns this notion into a fixed structural feature available to a learnable message function, rather than an exact equivalence test, consistent with the fact, noted above, that it does not encode the full multiset and may coincide for different local incidence patterns.*

Example 4.5. Assume $K \geq 3$. Take the triangle illustration of Section 3.1 in its simplest concrete form: $H_1 : \mathcal{E}_1 = \{\{a, b, c\}\}$ and $H_2 : \mathcal{E}_2 = \{\{a, b\}, \{a, c\}, \{b, c\}\}$, which share the same clique expansion. For the pair (a, b) , the first hypergraph gives a cardinality pattern corresponding to a hyperedge of size 3, while the second gives a cardinality pattern corresponding to a hyperedge of size 2. Hence

$$\chi_{ab}^{H_1} = \mathbf{c}_3, \quad \chi_{ab}^{H_2} = \mathbf{c}_2.$$

Thus, the proposed layer can distinguish these two hypergraphs despite their identical unweighted clique projection. This illustrates that χ_{vu} acts as a cardinality-sensitive refinement of pairwise adjacency, although not as a complete invariant of higher-order incidence structure.

We use a fixed one-hot cardinality encoding rather than a separate learnable cardinality embedding, keeping the structural information exposed to the model explicit. Since concatenating χ_{vu} increases the input dimension of the message MLP, improvements over variants without χ_{vu} cannot by themselves be attributed solely to meaningful hypergraph structure. For this reason, the experiments include a shuffled- χ control, obtained by globally shuffling the rows of χ across directed message-passing edges. This preserves the input dimension and the global multiset of descriptor rows, while destroying their assignment to the original vertex pairs and, in general, the symmetry $\chi_{vu} = \chi_{uv}$. The control therefore tests whether the model benefits from the structured descriptor rather than merely from the additional input dimension, but it does not isolate the effect of the pairwise assignment alone.

4.2.4. The EO-Pattern MPNN layer

Each vertex $v \in V$ carries a hidden representation

$$h_v^{(\ell)} \in \mathbb{R}^{d_\ell}$$

at layer ℓ . The goal of one message passing layer is to update this representation by aggregating information from the hypergraph neighbors of v .

For each neighbor $u \in N_H(v)$, we construct a message using three pieces of information: the current representation of the target vertex v , the current representation of the source vertex u , and the cardinality-sensitive descriptor χ_{vu} summarizing the sizes of the hyperedges through which v and u are jointly incident. This gives a learnable message of the form

$$\psi^{(\ell)}\left(h_v^{(\ell)}, h_u^{(\ell)}, \chi_{vu}\right),$$

where $\psi^{(\ell)}$ may be implemented, for instance, as an MLP.

Definition 4.6 (EO-Pattern MPNN layer). *The EO-Pattern MPNN layer maps hidden representations $\{h_v^{(\ell)}\}_{v \in V}$ to $\{h_v^{(\ell+1)}\}_{v \in V}$ via the following update rule for each $v \in V$:*

$$m_v^{(\ell+1)} = \sum_{u \in N_H(v)} P^{\text{EE}}(v, u) \psi^{(\ell)}\left(h_v^{(\ell)}, h_u^{(\ell)}, \chi_{vu}\right),$$

$$h_v^{(\ell+1)} = \phi^{(\ell)}\left(h_v^{(\ell)}, m_v^{(\ell+1)}\right),$$

where $\psi^{(\ell)}$ and $\phi^{(\ell)}$ are learnable functions of compatible dimensions.

A simple concrete choice is

$$h_v^{(\ell+1)} = \text{MLP}^{(\ell)}\left(h_v^{(\ell)} \parallel m_v^{(\ell+1)}\right),$$

where $\parallel$ denotes concatenation.

This architecture has the standard message/aggregate/update structure of a Gilmer et al. (2017) MPNN. However, the interaction between v and u is not determined only by their adjacency in the induced pairwise graph. By Proposition 4.2, $P^{\text{EE}}(v, u)$ is the transition probability from v to u in the equal-edges hypergraph random walk; in the row-wise aggregation above, the same coefficient weights the message that v receives from u . The message function is conditioned on the cardinality-pattern descriptor χ_{vu} , which allows the message to depend on the cardinality pattern of the hyperedges shared by v and u . Hence, the layer uses the induced neighborhood as the support of aggregation, but the propagation rule itself remains sensitive to the hypergraph structure that generated that neighborhood.

4.2.5. Relation to relational message passing

Section 4.2 already contrasted this construction with the relational-structure framework of Taha et al. (2025); here we make that contrast precise. In that framework, a higher-order object is represented as a relational structure

$$R = (S, R_1, \dots, R_k),$$

where S is a finite set of entities and each $R_i \subseteq S^{n_i}$ is an n_i -ary relation, that is, a set of ordered n_i -tuples of entities. The framework then associates shift operators with these relations (playing a role analogous to the propagation operators P_{prop} of Remark 2.10, but defined relation-by-relation rather than as a single hypergraph-level kernel). These shift operators are aggregated into influence matrices, which in turn define an influence graph: a directed weighted graph on the entity set S , with an edge $\tau \rightarrow \sigma$ whenever the aggregated influence of τ on σ is non-zero. This influence graph is then used to analyze information propagation and oversquashing.

The present construction does not use a relational structure as its starting point. It starts directly from the hypergraph incidence pattern. The message passing rule is determined by $\mathcal{E}(v, u)$, $|e|$, P^{EE} , and χ_{vu} . As with many finite message passing schemes, this model could plausibly be encoded, after the fact, inside a sufficiently rich relational framework. The point is not that such an encoding is impossible, but that it would not be the conceptual origin of the architecture.

Thus, the distinction is not that the proposed layer is impossible to relationalize, but that the primitive quantities defining propagation are computed directly from hypergraph incidence and hyperedge cardinalities.

4.2.6. Permutation equivariance

Proposition 4.7 (Permutation equivariance). *The proposed layer is permutation equivariant (Definition 2.8) with respect to relabelings of the vertex set.*

Proof. Let $\pi : V \rightarrow V$ be a permutation of the vertices, and let πH be the relabeled hypergraph with hyperedges

$$\pi \mathcal{E} := (\pi(e_\alpha))_{\alpha \in A} \quad \pi(e) := \{\pi(v) : v \in e\}.$$

Since π is injective, $\pi v \in \pi(e)$ if and only if $v \in e$ (and likewise for u), so a hyperedge of πH contains both πv and πu exactly when its preimage under π contains both v and u . Hence $e \mapsto \pi(e)$ is a bijection between $\mathcal{E}_H(v, u)$ and $\mathcal{E}_{\pi H}(\pi v, \pi u)$:

$$\mathcal{E}_{\pi H}(\pi v, \pi u) = (\pi(e_\alpha))_{\alpha \in A: v, u \in e_\alpha}$$

The same injectivity gives $|\pi(e)| = |e|$, and applying the bijection to $\mathcal{E}_H(v)$ likewise yields $d_{\pi H}(\pi v) = d_H(v)$. Since P^{EE} and χ depend on $\mathcal{E}_H(v, u)$ only through the cardinalities $|e|$ summed over this index set, substituting these facts into their defining formulas gives

$$P_{\pi H}^{\text{EE}}(\pi v, \pi u) = P_H^{\text{EE}}(v, u),$$

$$\chi_{\pi v, \pi u}^{\pi H} = \chi_{vu}^H.$$

The same bijection also gives

$$N_{\pi H}(\pi v) = \{\pi u : u \in N_H(v)\}.$$

To verify Definition 2.8 for this layer, take input hidden states that are themselves consistently relabeled,

$$h_{\pi v}^{(\ell)}(\pi H) = h_v^{(\ell)}(H) \quad \forall v \in V,$$

and check that the layer maps them to outputs related in the same way.

Since the message function $\psi^{(\ell)}$ and the update function $\phi^{(\ell)}$ are shared across all vertices, and since aggregation is performed by summation over the unordered neighborhood of each vertex, the resulting layer satisfies

$$h_{\pi v}^{(\ell+1)}(\pi H) = h_v^{(\ell+1)}(H).$$

Therefore, the layer is vertex permutation equivariant. $\square$

4.2.7. Computational cost

Proposition 4.8 (Computational cost). *For this complexity analysis, singleton hyperedges are assumed to have already been discarded, since they do not contribute to the proposed construction.*

Let

$$S_H := \sum_{e \in \mathcal{E}} |e|(|e| - 1)$$

be the number of ordered vertex pairs generated by the hyperedges, and let

$$M_H := \sum_{v \in V} |N_H(v)|$$

be the number of directed edges in the clique-expanded support. Then the quantities $P^{\text{EE}}(v, u)$ and χ_{vu} can be precomputed in time $O(S_H + M_H(K - 1))$ and stored in $O(M_H(K - 1))$ memory. In particular, for fixed K , the preprocessing time is $O(S_H)$. Once these quantities are precomputed, one forward pass of the proposed layer has the same aggregation support as a clique-expansion MPNN, namely M_H directed pairwise interactions.

Proof. For each hyperedge $e \in \mathcal{E}$, the construction updates the quantities associated with all ordered pairs (v, u) such that $v, u \in e$ and $v \neq u$. The number of such ordered pairs is $|e|(|e| - 1)$. Thus, the total number of pair occurrences processed during the accumulation pass is

$$\sum_{e \in \mathcal{E}} |e|(|e| - 1) = S_H.$$

Each pair occurrence requires only an $O(1)$ update: since $\mathbf{c}_{|e|}$ is one-hot, the contribution $\frac{1}{|e|-1} \mathbf{c}_{|e|}$ increments a single cardinality bin of the unnormalized descriptor accumulator, together with the scalar accumulator Z_{vu} . The $K - 1$ factor appears only when the dense descriptors are materialized and normalized for the M_H distinct pairs.

These updates accumulate the scalar contributions defining $P^{\text{EE}}(v, u)$ and the corresponding cardinality-bin contributions defining χ_{vu} . Within the same pass, $Z_{vu} = \sum_{e \in \mathcal{E}(v, u)} \frac{1}{|e|-1}$ is obtained automatically, since it is the same sum as the numerator of $P^{\text{EE}}(v, u)$. The degree $d_H(v) = |\mathcal{E}(v)|$ requires a separate count, incrementing once per hyperedge containing v rather than once per ordered pair (to avoid overcounting by a factor of $|e| - 1$); this costs $O(\sum_{e \in \mathcal{E}} |e|)$, which is $O(S_H)$, since $|e| \leq |e|(|e| - 1)$ for $|e| \geq 2$.

The sets $N_H(v)$ are obtained as a byproduct of the same pass: each pair occurrence triggers an $O(1)$ amortized insertion into the corresponding hash set, while repeated occurrences of the same pair do not create additional support edges. Thus, materializing all neighborhoods costs $O(S_H)$. Since every directed support edge is touched at least once, $M_H \leq S_H$; hence $M_H(K - 1) = O(S_H)$ when K is fixed. Normalizing $P^{\text{EE}}(v, u)$ costs $O(M_H)$, while normalizing and storing the dense descriptors χ_{vu} costs $O(M_H(K - 1))$. Hence the total preprocessing cost is $O(S_H + M_H(K - 1))$, which reduces to $O(S_H)$ when K is fixed.

After this preprocessing step, the layer aggregates messages only over pairs (v, u) with $u \in N_H(v)$. Therefore the number of message evaluations in one forward pass is

$$\sum_{v \in V} |N_H(v)| = M_H,$$

which is exactly the number of directed edges in the clique-expanded support. The additional hypergraph information appears in the precomputed edge weights and edge descriptors, not in a larger aggregation support. $\square$

The proposed layer is deliberately restricted to the minimal vertex-centered construction. Possible extensions involving additional propagation mechanisms or geometric information are discussed in Section 6.

5. Experiments and Validation

The previous section introduced a hypergraph-derived message passing layer whose purpose is to enrich a vertex-centered MPNN with information computed from the original hypergraph incidence structure. The experimental part of the thesis is designed to test this idea in a controlled way. The main question is not only whether the proposed layer improves numerical performance on standard benchmarks, but whether the quantities used by the layer, namely the equal-edges propagation kernel P^{EE} and the cardinality-pattern descriptor χ_{vu} , can make structural distinctions that are invisible to an ordinary clique-expanded graph.

The evaluation is therefore organized around three complementary questions:

1. Can the proposed layer distinguish hypergraphs that induce the same clique expansion but differ in their higher-order incidence structure? This is studied through a synthetic classification task specifically designed to isolate the information lost by clique expansion.
2. Does the same mechanism provide an advantage on real-world hypergraph benchmarks, where the labels may or may not depend on such cardinality-sensitive information? This is tested on co-citation datasets using clique-based graph baselines and hypergraph-derived variants.
3. When both P^{EE} and χ_{vu} are available, do they provide complementary information, or is one of them sufficient for the tasks considered?

Additional dataset statistics, training settings, and evaluation conventions are provided in Appendix A.

5.1. Synthetic Hypergraph Classification Task

We evaluate our EO-Pattern MPNN on a synthetic hypergraph classification task designed to isolate the effect of hyperedge cardinality. Each example consists of a pair of hypergraphs built on the same base graph: one class (“big”) uses the sampled cliques as hyperedges, while the other class (“small”) consists of the distinct vertex pairs contained in those hyperedges, that is, their unweighted clique expansion represented as a 2-uniform hypergraph. By construction, the two hypergraphs have identical clique expansions, so any model relying only on the graph projection (e.g. standard GCN or GIN) receives identical inputs for both classes. Thus graph-based baselines provably perform at chance (50%). In contrast, our EO-Pattern layer incorporates the cardinality descriptor χ (the sizes of shared hyperedges) in its message function, enabling it to distinguish the classes. We generate 300 random base graphs (with clique sizes and overlaps as in our configuration) and split them by base graph into training, validation and test sets (no graph appears in more than one split). The exact generation and splitting procedure is recorded in Appendix A.2.

We compare several models: two standard graph neural networks applied to the clique-expansion of each hypergraph (Clique-GCN, Kipf and Welling (2017); and Clique-GIN, Xu et al. (2019)), and ablations of our EO-Pattern architecture. In particular: (A) a uniform pairwise MPNN baseline with the same vertex update architecture as EO-Pattern, but with both hypergraph-derived ingredients removed, so that it aggregates uniformly over adjacent vertices and does not use P^{EE} or χ ; (B) EO-Pattern using the equal-edges propagation kernel P^{EE} , but ignoring χ ; (C) using only the χ descriptor with uniform propagation weights; (D) the full EO-Pattern MPNN, using both weights and χ ; and (E) the full model with the rows of χ globally shuffled

across directed message-passing edges as a negative control. All models are implemented as 2-layer MPNNs (Gilmer et al. (2017)) with 64 hidden units and trained for up to 300 epochs (with early stopping), and the dataset and split are fixed, while the five random seeds vary the model initialization and, for model (E), the random permutation of χ . Table 1 reports the test accuracy and macro-F1 score⁴ for each model (mean $\pm$ std).

Table 1: test performance on the synthetic hypergraph classification task (mean $\pm$ standard deviation over five run seeds). Best results are shown in bold.

Model	Accuracy (%)	Macro-F1 (%)
Clique-GCN (graph MPNN)	50.0 $\pm$ 0.0	33.3 $\pm$ 0.0
Clique-GIN (graph MPNN)	50.0 $\pm$ 0.0	33.3 $\pm$ 0.0
(A) Uniform EO baseline	50.0 $\pm$ 0.0	33.3 $\pm$ 0.0
(B) P^{EE} only	50.0 $\pm$ 0.0	33.3 $\pm$ 0.0
(C) χ only	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0
(D) EO-Pattern (full model)	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0
(E) Shuffled χ	50.0 $\pm$ 0.0	33.3 $\pm$ 0.0

The results validate the intended mechanism in this controlled construction. All models that do not use the original, unshuffled descriptor χ_{vu} (the clique-based GCN/GIN and variants (A), (B), and (E)) score 50% accuracy, corresponding to chance-level performance. By contrast, the EO-Pattern variants that use the unshuffled descriptor χ ((C) and (D)) achieve 100% accuracy and macro-F1, perfectly separating the classes. This confirms that, in this controlled twin construction, the unshuffled cardinality information encoded by χ_{vu} provides the usable discriminating signal: removing or shuffling the descriptor returns performance to chance. In summary, the synthetic experiment demonstrates that EO-Pattern MPNN can exploit cardinality-sensitive hypergraph-derived connectivity information that is invisible to unweighted clique-expansion baselines.

It is worth making precise in what sense this result is, and is not, about Weisfeiler-Leman expressivity. The chance-level behavior of Clique-GCN and Clique-GIN is not a 1-WL limitation in the usual sense of two non-isomorphic graphs that 1-WL fails to separate. By construction, the “big” and “small” classes have identical clique expansions: as attributed graphs, they are the same object, so any function of the clique-expanded graph alone, regardless of its expressive power, necessarily returns the same output on both classes. In particular, neither a GIN matching the discriminative power of 1-WL nor any k -WL procedure can distinguish the members of a twin pair. Consequently, any classifier based only on these representations is bounded at 50% accuracy on the balanced paired dataset: the separating information has been erased by graphification, not merely left undetected by a weak refinement. The EO-Pattern layer escapes this ceiling not by climbing the WL hierarchy, but by changing the input: it reads the hyperedge cardinalities through χ_{vu} before they are collapsed by clique expansion. This illustrates the orthogonality anticipated in the discussion following Definition 2.14: the k -WL hierarchy increases expressivity on a fixed graph domain, whereas the EO-Pattern layer enriches the pairwise domain with hypergraph-derived edge information while retaining a 1-WL-type vertex-centered aggregation.

⁴See Appendix A.4 for the role of each metric in this experiment.

5.2. Real-World Benchmark: Co-citation Node Classification

The synthetic experiment of Section 5.1 establishes that the EO-Pattern layer can access structure that is provably lost under clique expansion. We now test whether this advantage transfers to real-world data: node classification on the Cora and Citeseer co-citation hypergraphs (inspired by Yadati et al. (2019)), in which each hyperedge consists of the in-dataset papers cited by a common source paper. We use independently generated, class-stratified 50%/25%/25% train/val/test splits, one per seed, following the train/validation/test proportions used by Chien et al. (2022), and report test accuracy as mean $\pm$ std.

The incidence matrices of the co-citation hypergraphs were constructed using **TopoNetX**, the topological-domain construction package from the **TopoX** suite of Hajij et al. (2024). In the implementation, each non-singleton citation hyperedge is added as a rank-one cell. These cells are stored in a **ColoredHyperGraph**, from which `incidence_matrix(0,1)` produces the node-hyperedge incidence matrix. Since **TopoNetX** returns the incidence rows in its internal node order, they are reordered to match the canonical node indexing used by the feature and label matrices. The EO quantities P^{EE} and χ_{vu} are then computed from this incidence matrix. A **TopoNetX**-independent **SciPy** incidence builder is used both as a fallback and to validate the row-reordering step; the resulting incidence matrices are asserted to agree up to hyperedge-column ordering.

Nine models are evaluated: a feature-only MLP control, **Clique-GCN**, **Clique-GIN**, a mean-normalized **AllDeepSets**-style incidence baseline, and the five EO-Pattern ablations (A)-(E) described in Section 5.1. This in-house baseline applies an MLP before and after mean aggregation in both the node-to-hyperedge and hyperedge-to-node directions. It follows the two-stage Deep Sets design of Chien et al. (2022), but it is not a reimplement of their published **AllDeepSets** architecture, whose aggregation operator is a sum. It is evaluated with the same input projection, outer residual structure, dropout rate, training loop, splits, early-stopping criterion, and metrics as the other structural models. All structural node-classification models share the same per-layer residual update $h \leftarrow \text{ReLU}(\text{out}) + h$ (He et al., 2016), two layers, and hidden width 128. The feature-only MLP is kept as a non-residual control.

Table 2: node-classification accuracy (%) on Cora and Citeseer co-citation hypergraphs (mean $\pm$ std over 20 seeds). *all*: accuracy over all test nodes; *active*: accuracy restricted to nodes with $d_H(v) > 0$ (53% of Cora and 44% of Citeseer nodes are active).

Model	Cora		Citeseer	
	all	active	all	active
MLP (features only)	73.8 $\pm$ 1.4	75.1 $\pm$ 2.0	72.2 $\pm$ 0.9	74.9 $\pm$ 1.9
Clique-GCN	79.9 $\pm$ 1.5	85.0 $\pm$ 1.7	73.2 $\pm$ 1.4	76.3 $\pm$ 2.1
Clique-GIN	76.5 $\pm$ 1.4	82.8 $\pm$ 1.5	71.3 $\pm$ 1.0	73.9 $\pm$ 2.0
AllDeepSets (mean variant)	75.2 $\pm$ 1.5	76.5 $\pm$ 2.0	73.1 $\pm$ 1.0	75.9 $\pm$ 1.9
(A) Uniform EO baseline	76.2 $\pm$ 1.2	80.3 $\pm$ 1.4	72.4 $\pm$ 1.1	75.7 $\pm$ 1.6
(B) P^{EE} only	76.6 $\pm$ 1.3	80.9 $\pm$ 1.6	72.5 $\pm$ 1.1	76.0 $\pm$ 1.5
(C) χ only	76.2 $\pm$ 1.5	80.4 $\pm$ 2.2	72.6 $\pm$ 1.1	76.0 $\pm$ 1.8
(D) EO-Pattern (full model)	76.3 $\pm$ 1.4	80.8 $\pm$ 1.8	72.5 $\pm$ 0.9	76.2 $\pm$ 1.6
(E) Shuffled χ	76.6 $\pm$ 1.3	80.8 $\pm$ 1.9	72.6 $\pm$ 1.1	76.1 $\pm$ 1.6

Under this matched comparison, **CLIQUE-GCN** is clearly stronger than the EO variants on Cora. On Citeseer, their all-node accuracies are close, while their active-node accuracies are essentially tied. The co-citation data contain 96 duplicate occurrences among 1579 hyperedge columns on Cora and 75 among 1079 on Citeseer, corresponding to identical sets of co-cited papers induced by distinct source papers. These repetitions are retained as distinct hyperedges, consistent with the multiset convention of Definition 3.1. They leave the adjacency support $N_H(v)$, and hence the clique baselines and variant (A), unchanged. However, they affect the mean-normalized incidence baseline through the incidence structure and can alter $d_H(v)$, $P^{EE}(v, u)$, and χ_{vu} , and therefore variants (B)-(E). The conclusions below concern this multiplicity-aware representation.

The honest reading of Table 2 is a negative result for χ on co-citation data. On both datasets the shuffled- χ control (E) matches the full model (D), providing no evidence that the model exploits the structured organization of the cardinality descriptors. On Cora, variants (A)-(D) range from 76.2% to 76.6%. Relative to (A), the P^{EE} -only variant (B) gains 0.4 percentage points, while (C) and (D) differ by at most 0.1 points; the shuffled control (E) reaches 76.6%, slightly above (D). Thus, the structured organization of χ provides no observable benefit under this protocol, while P^{EE} produces only a small gain. The EO variants exceed the MLP by 2.4-2.8 points on average, but this difference cannot be attributed exclusively to neighborhood aggregation because the MLP is non-residual. Note, however, that variant (A) reaches 76.2, which is clearly below the residual **CLIQUE-GCN** result of 79.9. Thus, on Cora, the EO-Pattern aggregation operator is not competitive with the residual **CLIQUE-GCN** baseline. This should be separated from the ablation question addressed by the (A)-(E) ladder: whether P^{EE} and χ add value within the EO-Pattern family itself. The nearly flat ladder suggests that, on this dataset, they do not provide a clear additional gain. On Citeseer, all EO-Pattern variants lie within 0.5 percentage points of the MLP and show no clear improvement over it.

Another observation concerns node coverage: 47% of Cora nodes and 56% of Citeseer nodes satisfy $d_H(v) = 0$ and receive no messages from other nodes under any structural model. Consequently, all-node accuracy mixes structurally active nodes with nodes for which neighborhood propagation is unavailable, motivating the separate active-node results. This does not alter the comparative conclusion: **CLIQUE-GCN** remains clearly stronger on Cora, whereas the active-node results on Citeseer are tightly clustered. This reflects the co-citation hyperedge construction of Yadati et al. (2019): each hyperedge contains the in-dataset papers cited by a common source paper, and hyperedges with fewer than two members are dropped. A node is isolated precisely when it does not occur in any retained cited-paper set; its own outgoing citations do not by themselves make it incident to a hyperedge.

This does not contradict the result of Section 5.1; it sharpens the question. The failure on co-citation does not settle whether the EO quantities are ever useful for node classification. The following section constructs a case in which the normalized quantities P^{EE} and χ are jointly useful, and in which their combination identifies information that either quantity alone loses.

5.3. Joint Usefulness of P^{EE} and χ : A Constructive Gadget

The benchmark of Section 5.2 shows no observable benefit from χ and at most a small gain from P^{EE} on the co-citation data. A natural question is whether the two quantities are ever jointly load-bearing at all. We answer affirmatively with a constructive synthetic experiment: we design a node-classification task in which the informative neighbor is identifiable from the joint signature (P^{EE}, χ) , but not from either normalized quantity alone. The exact data-generation

and splitting procedure is recorded in Appendix A.2.

Each gadget attaches to a target vertex v three types of neighbor, distinguished by the way they connect to v :

- A single informative neighbor (Q), whose one-hot feature encodes the true label of v , connected through two size-3 hyperedges.
- m decoys of type P , whose one-hot features encode independently sampled random classes, each connected through one size-2 hyperedge.
- m decoys of type R , whose one-hot features encode independently sampled random classes, each connected through one size-3 hyperedge.

with neutral filler vertices completing the size-3 hyperedges. These filler vertices are also adjacent to v , but they carry no class-indicator feature: their feature vectors are zero before the addition of independent Gaussian noise with standard deviation 0.01. They serve only to complete the hyperedges. Writing $Z_{vu} = \sum_{e \in \mathcal{E}(v,u)} \frac{1}{|e|-1}$ for the equal-edges mass shared by v and u , the three neighbor types have the signatures shown in Table 3.

Table 3: structural signatures of the three neighbor types relative to the target vertex v .

Type	Shared hyperedge cardinality	Z_{vu}
P	2	1
Q	3	1
R	3	1/2

By construction, χ distinguishes P from $\{Q, R\}$ but collapses Q and R , whereas P^{EE} distinguishes R from $\{P, Q\}$ but collapses P and Q . Only their joint signature identifies the informative neighbor Q , which is the precise sense in which the experiment tests complementarity. We stress that the claim is relative to the normalized factorization (P^{EE}, χ) adopted by the layer: the unnormalized binned equal-edges mass, obtained by grouping the contributions to Z_{vu} by cardinality before division by Z_{vu} , would already yield distinct signatures $P = (1, 0)$, $Q = (0, 1)$, and $R = (0, 1/2)$. Therefore, the gadget does not prove that no single hypergraph-derived descriptor can solve the task; it rather shows that the two normalized quantities used by this layer encode complementary information.

The mean-normalized **AllDeepSets**-style baseline obtains the same 54.9% test accuracy in all ten runs, which coincides with the majority-class baseline for these stratified test sets and yields the reported zero standard deviation. Since this in-house variant was not tuned separately, this result should not be interpreted as an expressivity limitation of the **AllDeepSets** architecture of Chien et al. (2022).

The full model (D) reaches 79.5%, well above both the P^{EE} -only ablation (B, 68.1%) and the χ -only ablation (C, 64.2%). The important point is the observed superadditive pattern: P^{EE} alone adds +5.6 pp over the uniform EO baseline (A, 62.5%), χ alone adds only +1.7 pp, yet their combination adds +17.0 pp. The control (E) falls back to the P^{EE} -only level (68.6 $\approx$ EO-B 68.1), indicating that the improvement in (D) depends on the unshuffled structured descriptor acting jointly with P^{EE} , rather than merely on the additional input dimension or parameters associated with χ . Since the global shuffle also disrupts the local distribution and,

Table 4: test accuracy on the P/Q/R complementarity gadget (mean $\pm$ standard deviation over 10 seeds; 450 gadgets, $m = 4$ decoys per type, hidden width 64, and one layer). The best result is shown in bold.

Model	Accuracy (%)
MLP (features only)	54.2 ± 2.1
Clique-GCN	63.5 ± 3.8
Clique-GIN	63.3 ± 3.5
AllDeepSets (mean variant)	54.9 ± 0.0
(A) Uniform EO baseline	62.5 ± 3.9
(B) P^{EE} only	68.1 ± 3.2
(C) χ only	64.2 ± 4.6
(D) EO-Pattern (full model)	79.5 ± 3.0
(E) Shuffled χ	68.6 ± 3.5

in general, the symmetry of the descriptor, this comparison does not isolate the effect of the pairwise assignment alone.

Two points limit the scope of this result. First, because the informative neighbor Q is always present and truthful, even uniform aggregation retains a weak label signal. The relevant comparison is therefore the gap between (D) and the ablations, not the absolute accuracy. Conversely, although the joint signature identifies Q at the level of individual pairs, the implemented layer does not select neighbors explicitly: it weights and sums their messages. The informative contribution of Q is therefore mixed with those of the random decoys and neutral fillers, so structural identifiability does not imply perfect classification. Second, this is a constructive existence argument, not an empirical generalization claim: it shows that a regime of joint usefulness exists within this normalized decomposition, not that such regimes are common in natural data.

5.4. When Do the Hypergraph-Derived Quantities Carry Signal?

The three experiments form a consistent picture only if we can say precisely when P^{EE} and χ are informative and when they are inert. We do not have a full (necessary and sufficient) characterization: structural non-degeneracy alone is not sufficient, because usefulness depends on the task, specifically on the alignment of these quantities with the node features and target labels. We do, however, identify two clean structural conditions, derived directly from the definitions, under which χ loses pair-dependent variation or P^{EE} collapses to uniform neighborhood averaging. Recall from Sections 4.2.2 and 4.2.3 that, for an adjacent pair v, u ,

$$Z_{vu} = \sum_{e \in \mathcal{E}(v,u)} \frac{1}{|e| - 1}, \quad P^{\text{EE}}(v, u) = \frac{Z_{vu}}{d_H(v)}, \quad \chi_{vu} = \frac{1}{Z_{vu}} \sum_{e \in \mathcal{E}(v,u)} \frac{1}{|e| - 1} \mathbf{c}_{|e|},$$

where $\mathbf{c}_r$ is the fixed one-hot encoding of cardinality r into the $K - 1$ bins introduced in Section 4.2.3, with all cardinalities at least K assigned to the final bin.

Proposition 5.1 (Collapse of χ under cardinality-bin homogeneity). *Suppose that all hyperedges of H of size at least two belong to the same cardinality bin, so that $\mathbf{c}_{|e|} = \mathbf{c}$ for some fixed vector $\mathbf{c}$. Then $\chi_{vu} = \mathbf{c}$ for every adjacent pair (v, u) . In particular, if H is k -uniform on its hyperedges of size at least two (i.e., all such hyperedges have cardinality k), then $\chi_{vu} = \mathbf{c}_k$.*

Thus, χ carries no pair-dependent information within H . Consequently, if χ is non-constant across adjacent pairs, then H contains hyperedges belonging to at least two distinct cardinality bins.

Proof. For an adjacent pair (v, u) , $Z_{vu} > 0$. Since all shared hyperedges belong to the same cardinality bin, $\mathbf{c}_{|e|} = \mathbf{c}$ for every $e \in \mathcal{E}(v, u)$, so

$$\chi_{vu} = \frac{1}{Z_{vu}} \left(\sum_{e \in \mathcal{E}(v, u)} \frac{1}{|e| - 1} \right) \mathbf{c} = \frac{1}{Z_{vu}} Z_{vu} \mathbf{c} = \mathbf{c},$$

independently of the pair. The k -uniform case follows by taking $\mathbf{c} = \mathbf{c}_k$. The final statement follows by contraposition: if all non-singleton hyperedges belonged to one cardinality bin, then χ would be constant across adjacent pairs. $\square$

Proposition 5.2 (Collapse of P^{EE} to the uniform mean). *For any vertex v with $d_H(v) > 0$, the row $P^{\text{EE}}(v, \cdot)$ is uniform over the neighborhood $N_H(v)$ if and only if Z_{vu} is constant over $u \in N_H(v)$. In particular, if H is k -uniform on its hyperedges of size at least two and linear (no pair of distinct vertices belongs to more than one hyperedge, counting multiplicity), then, for every v with $d_H(v) > 0$ and every $u \in N_H(v)$, $P^{\text{EE}}(v, u) = 1/|N_H(v)|$. Hence, the kernel provides no pair-dependent weighting beyond uniform neighborhood averaging.*

Proof. If $Z_{vu} = z$ for all $u \in N_H(v)$, then $P^{\text{EE}}(v, u) = z/d_H(v)$ is constant across the row. Since the row is a probability distribution (Proposition 4.2) over $|N_H(v)|$ neighbors, the constant equals $1/|N_H(v)|$. Conversely, a uniform row forces $Z_{vu} = d_H(v)/|N_H(v)|$, constant. For the linear k -uniform case, each adjacent pair shares exactly one hyperedge of size k , so $Z_{vu} = \frac{1}{k-1}$. Linearity implies that the sets $e \setminus \{v\}$ associated with distinct hyperedges $e \in \mathcal{E}(v)$ are disjoint, so $|N_H(v)| = (k-1)d_H(v)$, and $P^{\text{EE}}(v, u) = \frac{1}{k-1}/d_H(v) = 1/|N_H(v)|$. $\square$

Propositions 5.1 and 5.2 identify structural conditions under which χ loses pair-dependent variation or P^{EE} collapses to uniform averaging. Together with feature degeneracy and task-dependent predictive alignment, these conditions organize the outcomes of the three experiments. Finally, isolated vertices form an additional vacuous case:

1. **Structural degeneracy.** If all non-singleton hyperedges belong to one cardinality bin, then χ is constant across adjacent pairs (Proposition 5.1). If Z_{vu} is constant over $u \in N_H(v)$, then $P^{\text{EE}}(v, \cdot)$ is uniform, with linear k -uniform hypergraphs as a particular case (Proposition 5.2). These are properties of the hypergraph alone, but they imply an absence of pair-dependent variation within H , not a complete absence of structural information across different hypergraphs.
2. **Feature degeneracy.** When node features are constant and no edge descriptor is used, every message is identical before weighting. Any row-stochastic weighting scheme therefore produces the same aggregate, making P^{EE} inert regardless of its values. This explains why ablation (B) remains at chance in Section 5.1, whereas χ , which enters the message function as an edge descriptor, can break the symmetry.
3. **Lack of predictive alignment.** Structural variation is not sufficient: it must also align with the node features and target labels. On the co-citation datasets, χ is non-constant

and P^{EE} is non-uniform on part of the hypergraph, but the structured organization of χ provides no observable benefit and P^{EE} provides at most a small gain under the present protocol (Section 5.2).

4. **Vacuous case (isolated vertices).** If $d_H(v) = 0$, then $N_H(v) = \emptyset$, no pairwise descriptor is available, and the aggregated message is zero. The EO-Pattern layer consequently reduces to a node-wise update. This vertex-level case motivates reporting active-node results separately in Section 5.2.

By contrast, the complementarity gadget of Section 5.3 avoids structural and feature degeneracy and deliberately aligns the joint signature (P^{EE}, χ) with the target label. Both quantities therefore contribute within the chosen normalized factorization. This establishes that the regime of joint usefulness is non-empty, while a general characterization of when structural variation aligns with features and labels remains open.

6. Discussion, Limitations, and Future Work

The experiments and Propositions 5.1 and 5.2 together support a precise and honest reading of the proposal. The EO-Pattern layer is well defined and permutation-equivariant, its cardinality descriptor provably accesses structure that clique expansion destroys (Section 5.1), and there exist tasks in which, within the layer’s normalized factorization, the equal-edges kernel and the descriptor provide complementary information through their joint signature (Section 5.3). On the co-citation benchmarks, however, the structured organization of χ provides no observable benefit, while P^{EE} yields at most a small gain beyond uniform neighborhood aggregation (Section 5.2). The propositions explain why the positive controlled results and the weak real-world gains are not in tension: structural non-degeneracy is present on the co-citation datasets, but usefulness additionally requires task-dependent alignment with the node features and target labels, which cannot be established from structure alone.

Two limitations should be stated plainly. First, the positive results are obtained on controlled constructions designed to isolate a mechanism, and they establish existence, not prevalence: they do not show that hypergraph-derived cardinality signal is common in natural data. Second, the EO-Pattern aggregation operator is not consistently competitive with the residual clique-expansion GCN: it performs substantially worse on Cora and provides no clear advantage on Citeseer. The proposal is therefore best read as a minimal, interpretable probe of what clique expansions forget rather than as a drop-in replacement for existing hypergraph neural networks.

This identifies the central open problem: finding sufficient conditions under which the EO-Pattern quantities improve on simpler aggregation rules. As a first measurable direction, let (V, U) be sampled uniformly from the ordered adjacent pairs $(v, u) : u \in N_H(v)$, and define the random variables $A := \mathbb{1}_{y_V = y_U}$, $R := P^{\text{EE}}(V, U)$, $C := \chi_{VU}$, and $S := (R, C)$. One can then estimate the joint mutual information $I(S; A)$, together with the conditional quantities $I(C; A \mid R)$ and $I(R; A \mid C)$, to assess the incremental alignment contributed by each component. Near-zero values would indicate weak pairwise label alignment, but would not prove inertness; conversely, high values would identify promising datasets without guaranteeing improved performance. Testing whether these statistics predict the observed ablation gains across datasets would turn the present post-hoc interpretation into a falsifiable empirical hypothesis.

Three further directions follow naturally. First, evaluation in the label-scarce regime. The sparse semi-supervised splits of HyperGCN (Yadati et al., 2019) (around twenty labels per class) provide a natural complement to the data-rich 50%/25%/25% protocol used here and would test whether structural propagation becomes more useful when supervision is limited.

Second, the base layer deliberately avoids additional mechanisms such as hyperedge hidden states, hyperedge-to-hyperedge propagation, quotient-level summaries, or curvature gates. These extensions may be useful, but they are not necessary to define the core vertex-centered hypergraph-derived layer. A natural extension inspired by Eidi and Otter (2025) would be to include an Ollivier-Ricci curvature gate. If $\kappa_{\text{EE}}(v, u)$ denotes the equal-edges Ollivier-Ricci curvature associated with the pair (v, u) , one could define the following curvature-gated kernel for $d_H(v) > 0$:

$$\tilde{P}^{\text{EE}}(v, u) = \frac{P^{\text{EE}}(v, u) g(\kappa_{\text{EE}}(v, u))}{\sum_{w \in N_H(v)} P^{\text{EE}}(v, w) g(\kappa_{\text{EE}}(v, w))},$$

where $g : \mathbb{R} \rightarrow \mathbb{R}_{>0}$ is either fixed or learnable. The normalization preserves row-stochasticity.

This would make the layer sensitive not only to local hyperedge cardinalities, but also to the local geometry induced by optimal transport. Such a mechanism may be effective in regimes where the present quantities cannot. Nevertheless, this curvature-gated version should be considered an extension, not part of the minimal model. The purpose of the present construction is to isolate a mathematically well-defined and implementable vertex-centered layer before introducing more sophisticated mechanisms.

Third, the benchmark could be strengthened by including established incidence-based hypergraph neural networks such as HGNN (Feng et al., 2019), the published AllDeepSets architecture, or AllSetTransformer (Chien et al., 2022); the present experiments include only an in-house mean-normalized Deep Sets-style incidence baseline, and broader comparison is deferred to future work.

7. Conclusions

This thesis pursued two connected aims: to provide a self-contained introduction to higher-order message passing and to formulate and experimentally validate a concrete hypergraph-derived layer. The two parts were designed to complement each other, with the survey providing the framework for the proposed model.

On the expository side, we assembled a self-contained account of how a neural network on relational data is built from carrier objects, a neighborhood notion, and a propagation rule, and of how this template survives the passage from graphs to hypergraphs, simplicial, cellular, and combinatorial complexes. We emphasized two recurring tensions that organize the field: the choice of propagation operator, made explicit through the random-walk viewpoint, where even a hypergraph admits no canonical walk; and the trade-off between reusing graph machinery through auxiliary expansions and preserving the higher-order information that those expansions may erase.

On the constructive side, we proposed the EO-Pattern layer, a vertex-centered message passing layer that keeps a pairwise aggregation support but conditions each message on two quantities computed directly from the hypergraph incidence structure: the equal-edges propagation kernel P^{EE} and the cardinality-pattern descriptor χ_{vu} . We showed that the layer is well defined, that its propagation weights sum to one at every vertex with $d_H(v) > 0$ (Proposition 4.2), that it is permutation-equivariant, and, for fixed cardinality cutoff K , computable at the same asymptotic cost as a clique-expansion MPNN. The guiding idea is the separation between the support of message passing, which remains pairwise, and the structure used to define the messages, which remains genuinely hypergraph-derived.

The experimental study was deliberately diagnostic rather than competitive. A synthetic twin construction confirmed that χ_{vu} recovers cardinality information that clique expansion provably destroys, a separation that, as we made precise, is not a Weisfeiler-Leman expressivity gap but a loss of information in the graphification itself. The co-citation benchmark then delivered an honest negative result for χ_{vu} , whose structured organization provides no observable benefit, while P^{EE} yields at most a small gain over uniform neighborhood aggregation. Finally, a constructive gadget exhibited a regime in which the joint signature (P^{EE}, χ_{vu}) uniquely identifies the informative neighbor and the full model substantially outperforms either single-component ablation. Propositions 5.1 and 5.2 identify structural regimes in which χ loses pair-dependent variation or P^{EE} collapses to uniform averaging. The subsequent analysis also identifies feature degeneracy and task-dependent alignment with node features and target labels as determinants of whether these quantities are useful.

The limitations discussed in Section 6 point to a central open problem: finding sufficient conditions under which the EO-Pattern quantities improve on simpler aggregation rules. As a first step, we proposed joint and conditional mutual-information statistics based on the signature (P^{EE}, χ) and label agreement as measurable diagnostics of predictive alignment. Whether these diagnostics predict ablation gains across datasets is a falsifiable empirical hypothesis, not a sufficient condition established here. Together with the label-scarce evaluation regime and the curvature-gated extension outlined there, this delineates a concrete agenda. More broadly, the tension that motivated the proposal, between intrinsically hypergraph-native propagation and the relationalization of higher-order domains into auxiliary graphs, remains the open conceptual question framing work beyond this thesis.

References

- Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2006.05205.
- Maciej Besta, Florian Scheidl, Lukas Gianinazzi, Grzegorz Kwasniewski, Shachar Klaiman, Jürgen Müller, and Torsten Hoefer. Demystifying higher-order graph neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 48(3):2544–2565, March 2026. doi: 10.1109/TPAMI.2025.3637114.
- Timoteo Carletti, Federico Battiston, Giulia Cencetti, and Duccio Fanelli. Random walks on hypergraphs. *Physical Review E*, 101(2):022308, 2020. arXiv:1911.06523.
- Eli Chien, Chao Pan, Jianhao Peng, and Olgica Milenkovic. You are allset: A multiset function framework for hypergraph neural networks. In *International Conference on Learning Representations*, 2022.
- Corinna Coupette, Sebastian Dalleiger, and Bastian Rieck. Ollivier-ricci curvature for hypergraphs: A unified framework. In *International Conference on Learning Representations*, 2023.
- Marzieh Eidi and Nina Otter. Geometric characterisation of structural and regular equivalences in undirected (hyper)graphs, 2025.
- Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. Hypergraph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 3558–3565, 2019.
- Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. arXiv:1704.01212.
- Mustafa Hajij, Ghada Zamzmi, Theodore Papamarkou, Nina Miolane, Aldo Guzmán-Sáenz, Karthikeyan Natesan Ramamurthy, Tolga Birdal, Tamal K. Dey, Soham Mukherjee, Shreyas N. Samaga, Neal Livesay, Robin Walters, Paul Rosen, and Michael T. Schaub. Topological deep learning: Going beyond graph data. *arXiv preprint*, 2023. arXiv:2206.00606v3.
- Mustafa Hajij, Mathilde Papillon, Florian Frantzen, et al. Topox: A suite of python packages for machine learning on topological domains. *Journal of Machine Learning Research*, 25(374): 1–8, 2024.
- Mustafa Hajij, Lennart Bastian, Sarah Osentoski, Hardik Kabaria, John L. Davenport, Sheik Dawood, Balaji Cherukuri, Joseph G. Kocheemoolayil, Nastaran Shahmansouri, Adrian Lew, Theodore Papamarkou, and Tolga Birdal. Copresheaf topological neural networks: A generalized deep learning framework. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- Glen Jeh and Jennifer Widom. Scaling personalized web search. In *Proceedings of the 12th International Conference on World Wide Web (WWW)*, pages 271–279. ACM, 2003.

- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1609.02907.
- Haggai Maron, Heli Ben-Hamu, Nadav Shamir, and Yaron Lipman. Invariant and equivariant graph networks. In *International Conference on Learning Representations (ICLR)*, 2019. Published as a conference paper at ICLR 2019.
- Mathilde Papillon, Guillermo Bernardez, Claudio Battiloro, and Nina Miolane. Topotune: A framework for generalized combinatorial complex neural networks. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 47924–47952. PMLR, 2025. URL <https://proceedings.mlr.press/v267/papillon25a.html>.
- Diaaeldin Taha, James Chapman, Marzieh Eidi, Karel Devriendt, and Guido Montúfar. Demystifying topological message-passing with relational structures: A case study on oversquashing in simplicial message-passing. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2506.06582v1.
- Terence Tao. What is good mathematics? *Bulletin of the American Mathematical Society*, 44(4):623–634, 2007. doi: 10.1090/S0273-0979-07-01168-8.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.10903.
- Ulrike von Luxburg. A tutorial on spectral clustering. *Statistics and Computing*, 17(4):395–416, 2007. arXiv:0711.0189.
- Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021. doi: 10.1109/TNNLS.2020.2978386. arXiv:1901.00596.
- Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1810.00826.
- Naganand Yadati, Madhav Nimishakavi, Prateek Yadav, Vikram Nitin, Anand Louis, and Partha Talukdar. HyperGCN: A new method of training graph convolutional networks on hypergraphs. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander J Smola. Deep sets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 3391–3401, 2017. arXiv:1703.06114.
- Dengyong Zhou, Jiayuan Huang, and Bernhard Schölkopf. Learning with hypergraphs: Clustering, classification, and embedding. In *Advances in Neural Information Processing Systems*, volume 19. MIT Press, 2006.

Appendix

A. Experimental Protocol and Reproducibility Details

This appendix summarizes the experimental conventions used throughout the three experiments.

A.1. Datasets and experimental settings

Table 5 summarizes the main structural statistics of the two co-citation benchmarks. All reported hyperedges have cardinality at least two. Duplicate occurrences count hyperedge columns beyond the first occurrence of each distinct vertex set. Active nodes satisfy $d_H(v) > 0$, while $M_H = \sum_{v \in V} |N_H(v)|$ counts ordered adjacent pairs and therefore equals the number of directed edges used by the clique-expansion and EO-Pattern models.

Table 5: structural statistics of the Cora and Citeseer co-citation hypergraphs.

Dataset	n	F	Classes	$ \mathcal{E} $	Duplicates	Active nodes	M_H
Cora	2708	1433	7	1579	96	1434 (53%)	8512
Citeseer	3312	3703	6	1079	75	1458 (44%)	8614

All models are trained in full-batch mode using Adam and cross-entropy loss. Early stopping is based on validation accuracy, and the parameters attaining the best validation accuracy are restored before test evaluation. Table 6 consolidates the remaining architecture and optimization settings. The maximum number of epochs is only an upper bound, since training may terminate earlier according to the patience parameter.

Table 6: architecture and optimization settings used in the three experiments.

Parameter	Synthetic	Co-citation benchmark	Complementarity
Message-passing layers	2	2	1
Hidden width	64	128	64
Dropout	0	0.5	0.3
Learning rate	10^{-2}	10^{-3}	10^{-2}
Weight decay	5×10^{-4}	5×10^{-4}	5×10^{-4}
Maximum epochs	300	200	200
Patience	40	40	30
Independent runs	5	20	10
Cardinality cutoff K	8	8	8

In the synthetic experiment, all models use a linear input projection, ReLU activations, two message-passing layers, mean pooling over the vertices of each graph, and a two-layer MLP classification head. No dropout or residual connections are used. In the two node-classification experiments, the structural models use a linear input projection and classification head together with the residual update $h \leftarrow \text{ReLU}(\text{out}) + h$ after each message-passing layer; the feature-only MLP is non-residual. All experiments are run on CPU. Reported standard deviations are population standard deviations across runs, corresponding to `ddof=0`.

A.2. Controlled data-generation details

Each of the 300 base graphs is generated as a union of k cliques, where k is sampled uniformly from $\{2, \dots, 5\}$ and each clique cardinality is sampled uniformly from $\{3, \dots, 6\}$. Successive cliques may reuse at most two previously generated vertices, with the remaining vertices being new. From the same clique family $\mathcal{C}$, the two hypergraphs are defined by

$$\mathcal{E}_{\text{big}} = \mathcal{C}, \quad \mathcal{E}_{\text{small}} = \bigcup_{C \in \mathcal{C}} \{\{u, v\} \subseteq C : u \neq v\}.$$

Duplicate hyperedges are removed. Both twins use the same constant 16-dimensional node features. Their clique expansions are explicitly verified to coincide. The resulting 600 examples are split by base graph in proportions 60%/20%/20%, ensuring that the two members of a twin pair always belong to the same split. The dataset and split seeds are both fixed to 0.

The reported configuration contains 450 disjoint P/Q/R gadgets with $m = 4$ decoys of each type, giving 7200 vertices and 4500 hyperedges. The informative neighbor Q receives the one-hot encoding of the target label, whereas the P - and R -type decoys receive independently sampled binary class features. Target and filler vertices receive the zero vector. Independent Gaussian noise with standard deviation 0.01 is added to every feature coordinate. The complete dataset is generated once using seed 12345. For each run, target vertices are split separately by class into 50% training, 25% validation, and 25% test sets.

A.3. Randomness and ablation protocol

A run seed controls model initialization and, when applicable, the data split, dropout masks, and the random permutation of χ used by model (E). Its precise role differs across experiments:

- **Synthetic.** The dataset and the train/validation/test split are fixed, with both the data and split seeds set to 0. For all models except (E), the five run seeds vary only the model initialization; for model (E), they also determine the permutation of χ . The paired construction forces any deterministic classifier based only on the common clique-expanded representation to attain 50 % accuracy, whereas the 100 % results of models (C) and (D) are empirical outcomes.
- **Benchmark.** Each of the twenty run seeds draws an independent per-class 50 %/25 %/25 % split and controls model initialization and dropout. For model (E), it also determines the permutation of χ . The reported variation therefore combines split and training variation and, for model (E), descriptor-permutation variation.
- **Complementarity gadget.** The hypergraph structure is fixed using `seed=12345`. Each of the ten run seeds draws an independent class-stratified split over target vertices and controls model initialization and dropout; for model (E), it also determines the permutation of χ .

The ablation study isolates the role of the two hypergraph-derived ingredients of the EO-Pattern layer. Model (A) uses only uniform clique-style neighborhood aggregation. Model (B) uses P^{EE} but ignores χ_{vu} . Model (C) uses χ_{vu} but replaces P^{EE} by uniform weights. Model (D) is the full EO-Pattern layer. Model (E) is a negative control in which χ_{vu} is randomly shuffled across adjacent pairs. Therefore, if (E) performs similarly to (D), the specific assignment of cardinality descriptors is not carrying useful signal for the task.

A.4. Accuracy and macro-F1

The main metric reported in the experiments is accuracy. For a test set $\mathcal{T}$, accuracy is defined as

$$\text{Acc} = \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \mathbb{1}\{\hat{y}_i = y_i\}.$$

It measures the fraction of correctly classified examples and is the standard metric used in node-classification benchmarks such as Cora and Citeseer.

In addition, we report macro-F1 in the synthetic twin experiment. For each class c , let TP_c , FP_c , and FN_c denote the corresponding numbers of true positives, false positives, and false negatives. Its F1 score is

$$F1_c = \frac{2\text{TP}_c}{2\text{TP}_c + \text{FP}_c + \text{FN}_c},$$

with $F1_c := 0$ when the denominator vanishes. Macro-F1 is the unweighted average over classes,

$$\text{MacroF1} = \frac{1}{C} \sum_{c=1}^C F1_c.$$

Unlike accuracy, macro-F1 penalizes classifiers that ignore one or more classes. This is relevant in the synthetic experiment because several baselines achieve chance-level accuracy by collapsing to essentially one class. In a balanced binary task, such a classifier may still obtain around 50% accuracy, but its macro-F1 drops to about 33.3%, revealing that it has not learned a meaningful class distinction.

For this reason, macro-F1 is most informative in the synthetic experiment, where the purpose is to verify whether the model truly distinguishes the two hypergraph classes rather than exploiting a degenerate prediction strategy. In the co-citation benchmark, accuracy is kept as the main reported metric because it is the conventional metric for Cora and Citeseer comparisons; macro-F1 is computed as an auxiliary diagnostic but does not change the qualitative conclusions. In the complementarity experiment, the task is a controlled and approximately balanced binary classification problem, so accuracy already captures the intended success criterion: whether the model can solve the P/Q/R gadget or not.